

"PrimeBid" - Forward Auctioning System

EECS 4413: Building e-Commerce Systems

**Software Design Document - Requirements
Documentation**

Team 5: Ascendants

Submission Date: February 6, 2026

Document Change Control:

Version	Date	Author(s)	Summary Of Changes
0.1	2026-02-03	Nathan Binu Edappilly 219317965	Initial creation of Software Design Document. Added project overview, 2 Activity diagrams for UC1 and UC2, and initial design decisions. Developed the Architecture section.
0.2	2026-02-03	Umer Shaikh 218 931 790	Created Sequence Diagrams for UC1, UC2, and UC3. Completed remaining Activity Diagrams (UC4–UC7) to ensure full use case coverage.
0.3	2026-02-03	Imaduddin Ahmed 219 569 029	Designed the UML Component Diagram and defined the Service Interfaces and API specifications.
0.4	2026-02-03	Pawanjot Randhawa 219 732 718	Added Project Plan components: Product Backlog, GANTT Chart, and Group Meeting Logs. Compiled the TDD (Test Driven Development) section.
1.0	2026-02-05	All Members	Final integration of all sections. Added AI Acknowledgment and References. Verified formatting and diagram consistency for final submission.

Table of Contents

1. Main Page
2. Introduction:
 - a. 2.1 Purpose
 - b. Overview
3. Major Design Decision
4. Sequence Diagrams
5. Activity Diagrams
6. Architecture
7. Activities Plan, Product Backlog, and Sprint Backlog
8. Test Driven Development (TDD)
9. Acknowledgment

Table of Contents - Milestone 2

- Abstract
- 10. Scope and Milestone 2 Constraints
 - 10.1 Scope
 - 10.2 Explicit Milestone Constraint
- 11. Deviations and Updates from Milestone 1
 - 11.1 Architecture and Service Boundaries
 - 11.2 Validation and Robustness Updates
 - 11.3 Consistency and Hypermedia
 - 11.4 Testing Strategy Evolution
 - 11.5 Clarification of "Real-Time" Scope and Behavior
 - 11.6 Deliverable 1 to Deliverable 2 Deviation Map
- 12. Alternative Architecture Comparison (Expanded)
- 13. Coupling/Cohesion Justification with Concrete Evidence
 - 13.1 Low Coupling Evidence
 - 13.2 High Cohesion Evidence
 - 13.3 Consistency Check (Architecture Narrative vs Diagram)
- 14. Design Patterns Used (4)
 - 14.1 Facade Pattern (Gateway)
 - 14.2 Repository Pattern (Persistence Boundary)
 - 14.3 Strategy Pattern (Bid Validation Rules)
 - 14.4 Observer Pattern (Auction End Side Effects)
- 15. Design Patterns Considered but Not Used (2)
 - 15.1 Abstract Factory (Considered)
 - 15.2 Manual Singleton (Considered)
- 16. Technology Choices and Justification
- 17. Use-Case and Test Completeness Matrix
- 18. Diagram Inventory
- 19. Diagram Walkthrough
- 20. AI Usage Acknowledgement (Advantages and Disadvantages)
 - 20.1 Scope of AI Use
 - 20.2 Human Oversight and Validation
 - 20.3 Advantages
 - 20.4 Limitations / Risks
 - 20.5 Mitigation Measures

21. Team Contribution

22. Team Meeting Logs

23. Mark Alignment

- 23.1 Implementation Mark (/80) Alignment
- 23.2 Report Content Mark (/15) Alignment
- 23.3 Report Writing Quality Mark (/5) Controls:

24. References

- 24.1 Academic & Course Materials
- 24.2 Technical Frameworks & Security
- 24.3 Design Patterns & Modeling
- 24.4 Project Artifacts & Scripts

2. Introduction

2.1 Purpose

The purpose of this Software Design Document (SDD) is to describe the structure, behavior, and key design decisions of the Forward Auctioning System. This document serves as a reference for developers, testers, and evaluators **by** clearly outlining the system requirements, architecture, and design constraints. It supports the implementation and validation of the system across all project milestones.

2.2 Overview

The Forward Auctioning System is a web-based application that enables users to participate in forward (highest-bidder) auctions. The system allows users to register, authenticate, browse auctioned items, place bids in real time, and complete payments if they win an auction. Sellers are able to upload items for auction by specifying item details, auction duration, and starting bid price.

The system enforces the following key rules and constraints:

- Only forward auctions are supported.
- All bids must be integer values and strictly higher than the current highest bid.
- Users may bid on only one item per login session.
- Only the winning bidder may proceed with payment once an auction ends.
- Final payment includes item price, shipping cost, and optional expedited shipping.
- Receipt and shipment information are displayed after payment confirmation.

The system is designed using a modular and service-oriented architecture, enabling clear separation of concerns between user management, auction handling, payment processing, and supporting services. This document provides an executive summary of the system requirements and the design approach used to implement them in a maintainable and extensible manner.

3. Major Design Decisions

3.1 Technology Stack:

1. **Front-End (Presentation Layer):** We use **HTML and CSS**, with optional **TypeScript**, to implement the web-based user interface and basic client-side interactions.
2. **Gateway and Application Layer:** We use **Java with the Spring Boot framework** to implement the Gateway (Façade) component and coordinate application workflows through RESTful APIs.

3. **Back-End Services (Business Logic Layer):** We use **Java and Spring Boot** to implement backend services, including IAM, Catalogue, Auction, Payment, and an Advanced Feature Service planned for Milestone 3.
4. **Data Layer:** We use **SQLite** as a lightweight relational database for persistence, with each service managing its own database to maintain low coupling and clear data ownership. **JDBC** is used for database access.
5. **Communication:** We use **RESTful HTTP with JSON payloads** for service communication. **WebSocket communication** may be used to support real-time bid updates.
6. **Deployment (Milestone 3):** We plan to use **Docker containers** to support service isolation and deployment for the advanced milestone.

3.2 Modularization and System Decomposition

The system is designed using a modular architecture to ensure high cohesion and low coupling between components. Each major responsibility of the Forward Auctioning System is encapsulated within a distinct module:

- **User Authentication Module:** Handles user sign-up, sign-in, password verification, and session validation.
- **Catalogue Module:** Manages auction items, including item descriptions, keywords, shipping costs, and auction durations.
- **Auction Module:** Controls the bidding process, enforces auction rules, tracks bids and bidders, and determines auction outcomes.
- **Payment Module:** Handles payment processing, validation of required payment details, and receipt generation.
- **User Interface Module:** Provides the web-based interface through which users interact with the system.

This separation ensures that changes in one module (e.g., payment processing) do not directly impact other modules (e.g., bidding logic), improving maintainability and testability.

3.3 Architectural Pattern Chosen

The system follows a **Layered Architecture**, consisting of the following layers:

1. **Presentation Layer:** User-facing pages for registration, login, browsing auctions, bidding, payment, and receipt display.
2. **Application Layer:** Controllers and services that coordinate use case execution and enforce system rules.
3. **Domain Layer:** Core business logic, including auction rules, bidding validation, and payment conditions.
4. **Data Layer:** Databases responsible for storing users, auction items, bids, and payment records.

This architecture improves maintainability, supports clear separation of concerns, and aligns with standard web application design practices taught in the course. A **Gateway service** is used as a Façade between the user interface and backend modules. It provides a single entry point for UI requests, orchestrates use case flows, and enforces system-wide constraints such as allowing users to bid on only one item per login session.

3.4 Architectural Pattern Considered but Not Chosen

A **fully distributed Microservices Architecture** was considered but not selected for this deliverable. While microservices offer benefits such as independent deployment and scalability, they introduce additional complexity in terms of inter-service communication, data consistency, and deployment infrastructure.

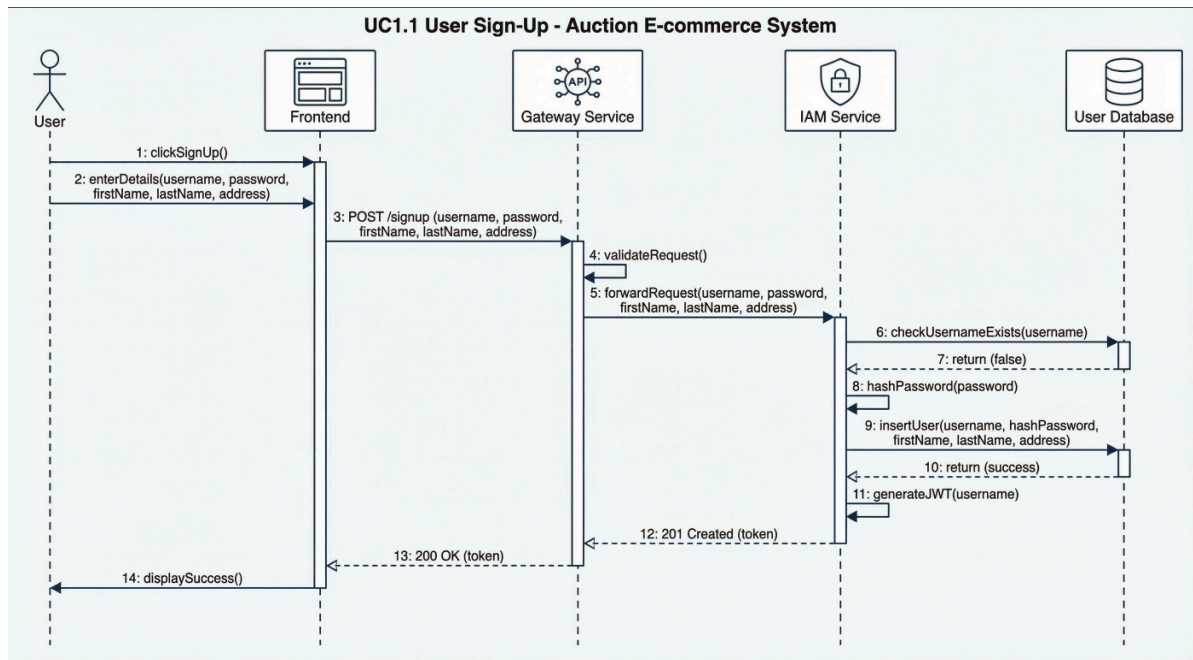
Given the academic scope of the project and the objectives of the early milestones, a layered modular architecture provides sufficient flexibility and clarity without unnecessary overhead. Elements of service-based and event-driven communication are reserved for later milestones as optional extensions.

3.5 Design Constraints and Assumptions

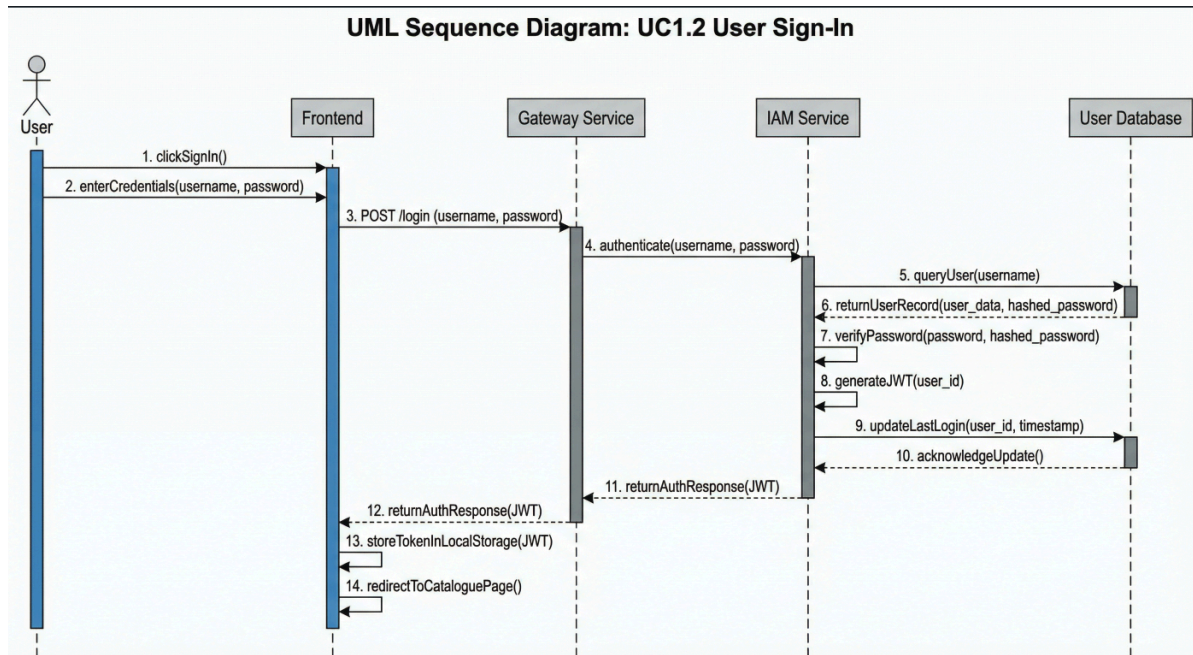
- The system uses server-side page refreshes rather than real-time push notifications.
- Payment details entered during testing are not real credit card data.
- Shipping dates are displayed as textual information (e.g., “Item will be shipped in XXX days”).
- Advanced AI-assisted or self-adaptive features are deferred to **Milestone 3** and are not implemented in earlier milestones.

4. Sequence Diagrams

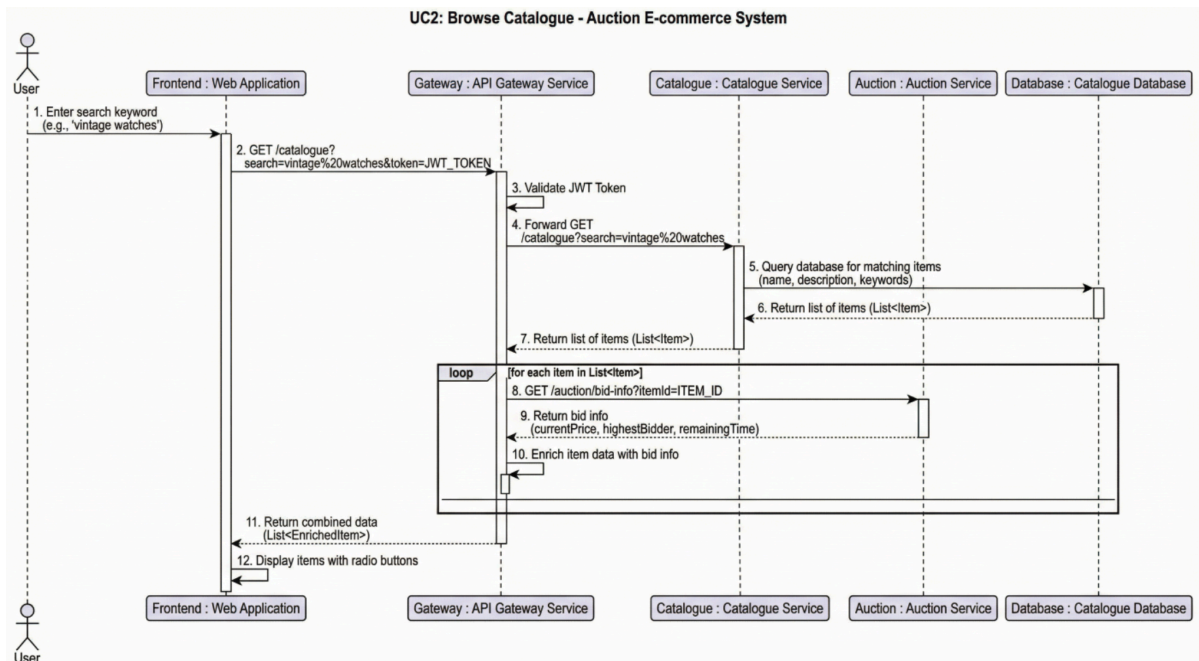
Sequence diagram for UC1.1 - Sign-Up:



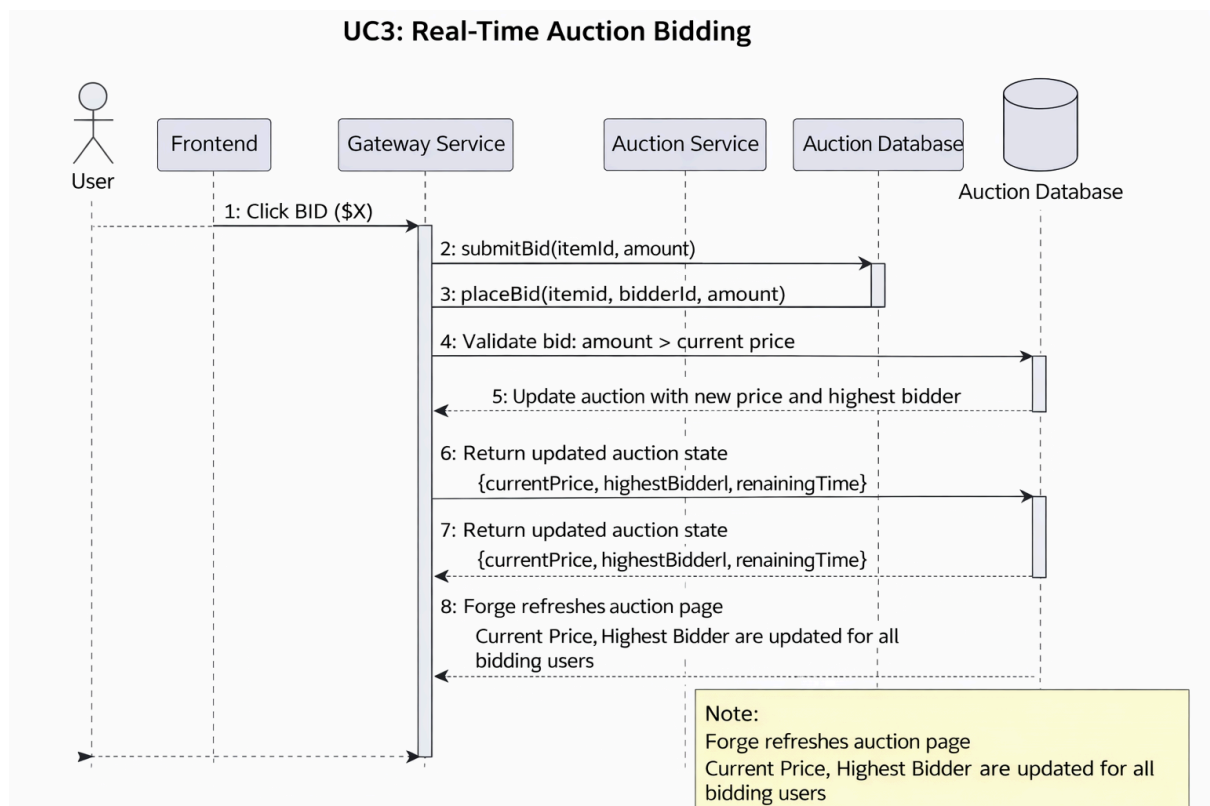
Sequence diagram for UC1.2 - Sign-In:



Sequence diagram for UC2:



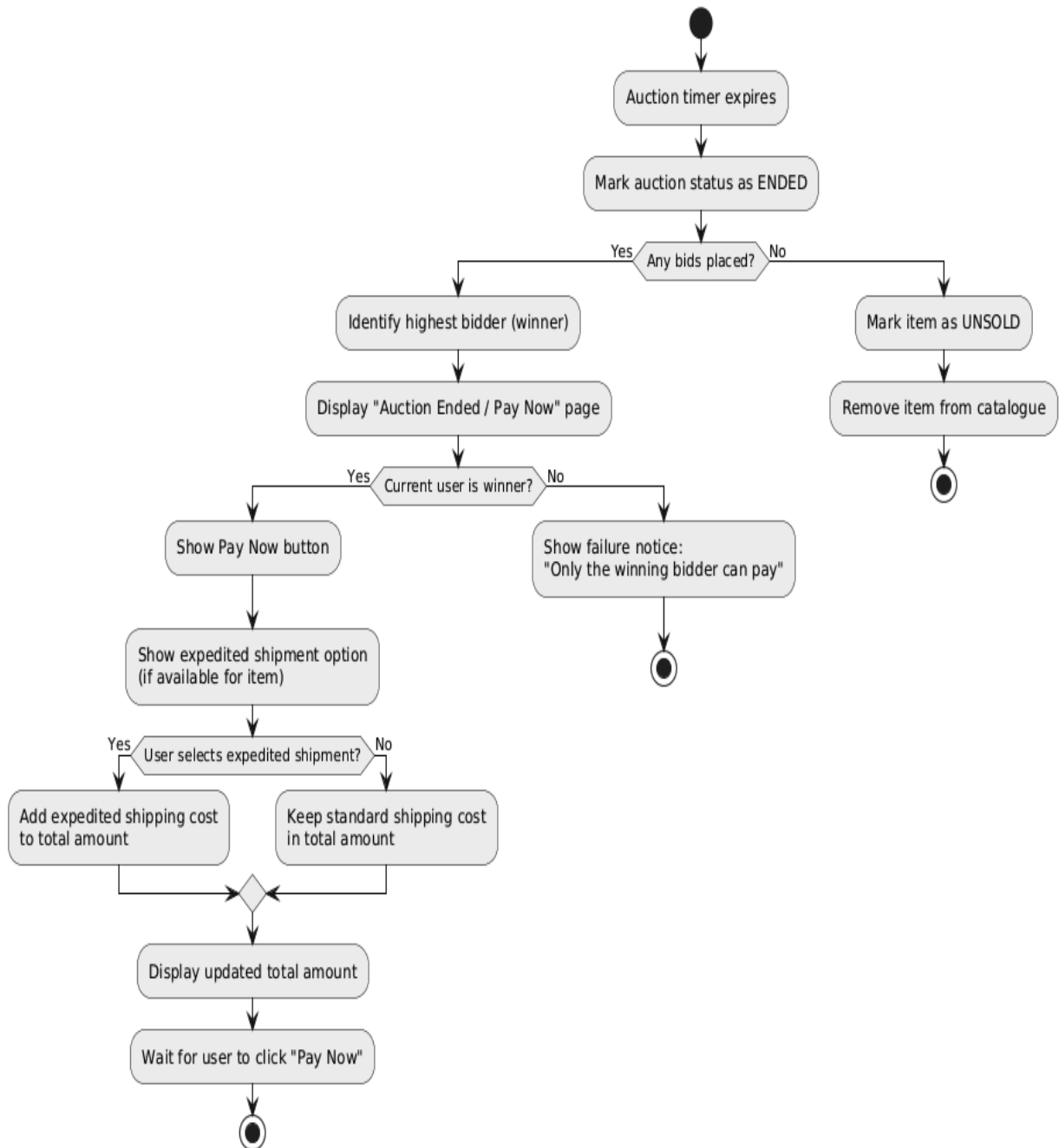
Sequence diagram for UC3:



5. Activity Diagrams

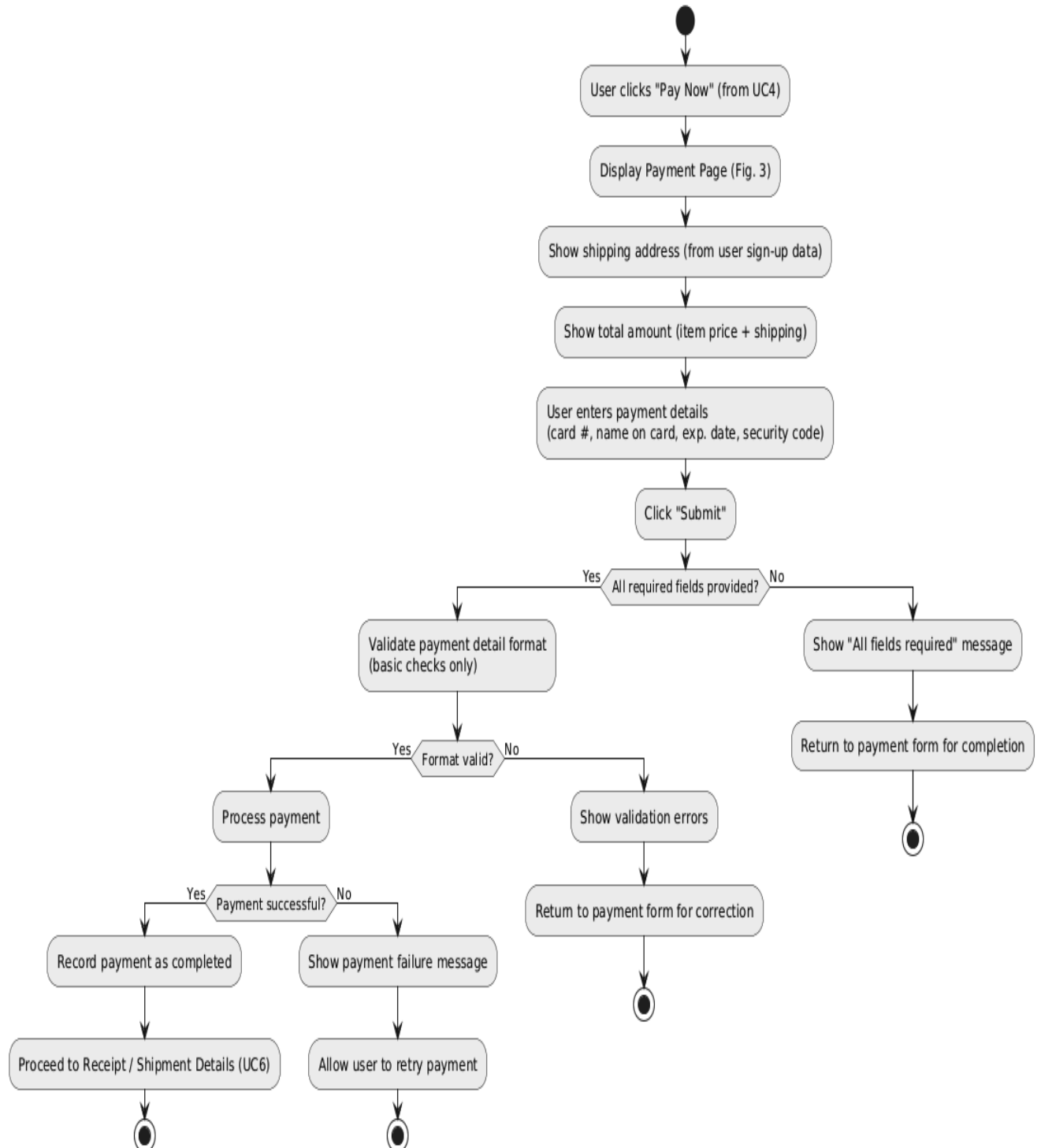
Activity Diagram for UC4:

AD-UC4: Auction Ended (Pay Now page + expedited option)



Activity Diagram for UC5:

AD-UC5: Payment (Winner submits card details)



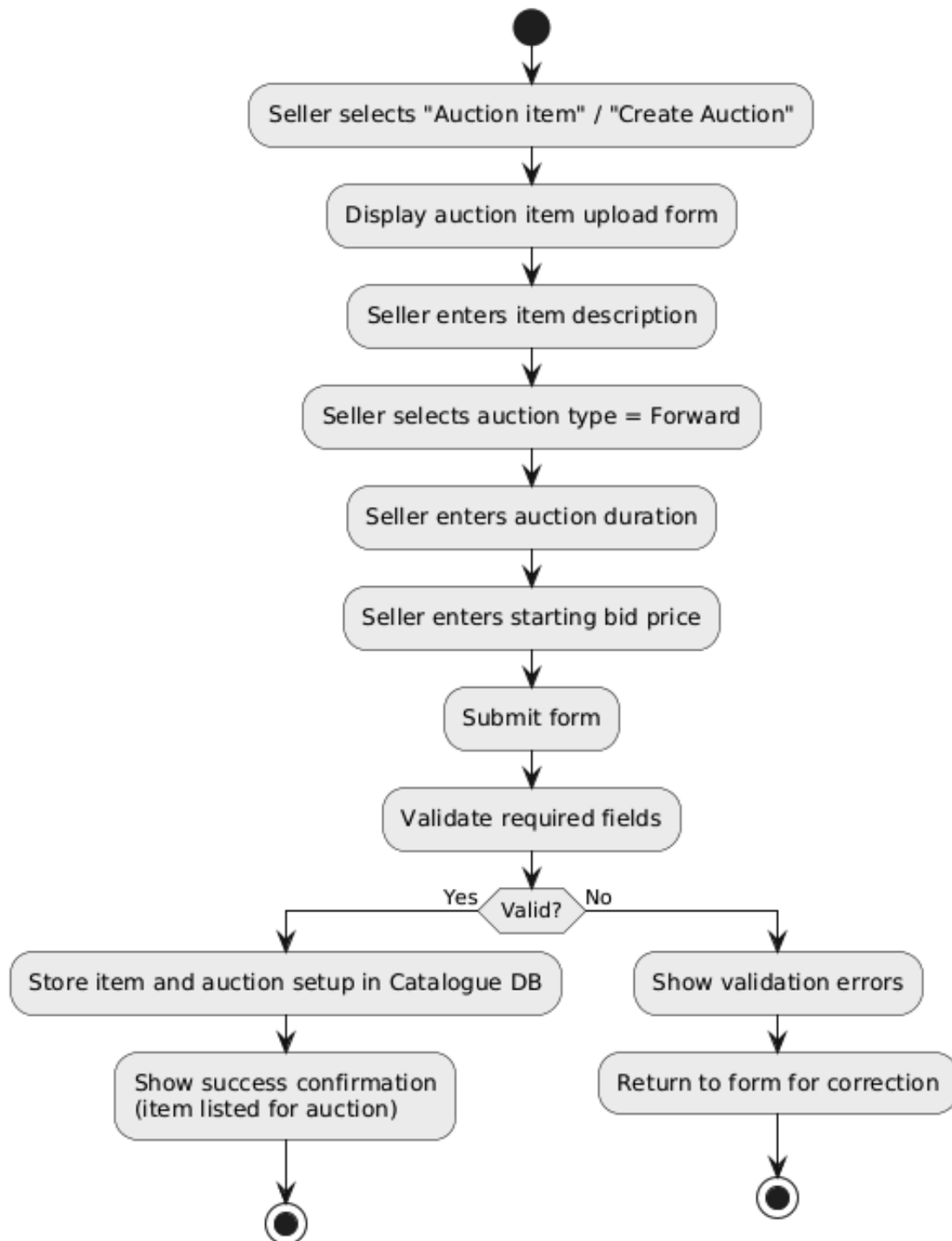
Activity Diagram for UC6:

AD-UC6: Receipt Page and Shipment Details

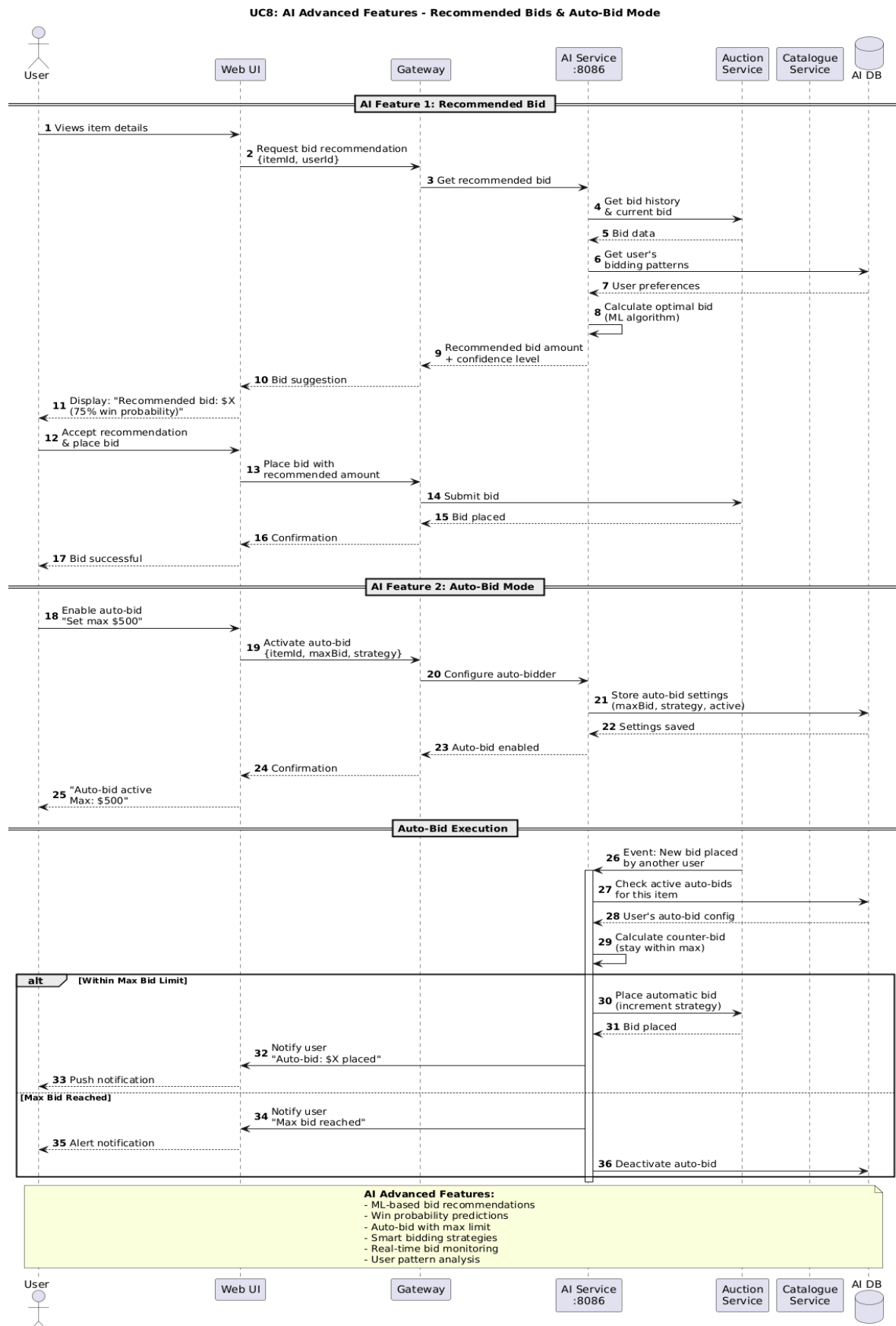


Activity Diagram for UC7:

AD-UC7: Auction Item (Seller Uploads Item for Forward Auction)



Activity Diagram for UC8:



6. Architecture Overview

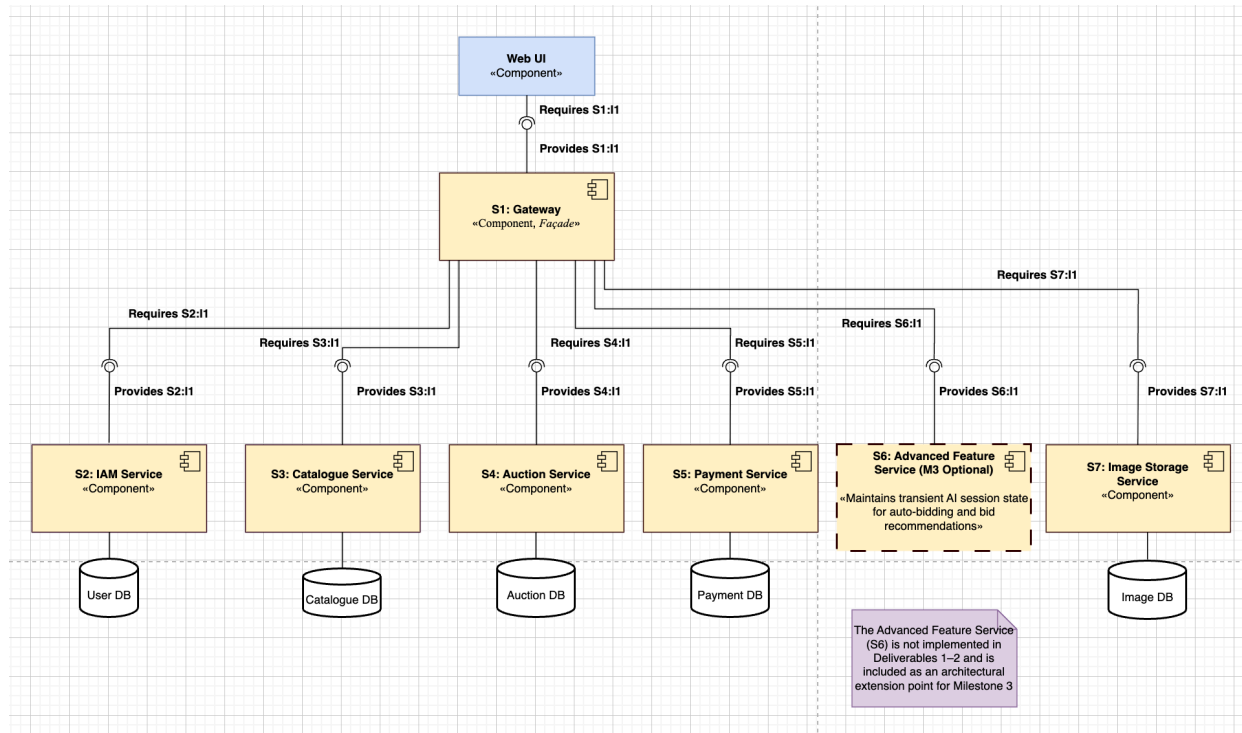
The Forward Auction System is implemented using a **layered architecture** to promote maintainability, modularity, and clarity. Each layer has a well-defined responsibility and communicates with adjacent layers through clearly defined interfaces.

Architectural Layers:

1. **Presentation Layer:** Handles user interaction through web-based pages for signing up, signing in, browsing auctions, bidding, payment, and receipt display.
2. **Application Layer:** Coordinates incoming requests, enforces workflow logic, and mediates communication between layers. This layer includes a **Gateway (Façade) component**, which serves as a single entry point for the user interface and orchestrates use case execution across backend services.
3. **Domain Layer:** Implements the core business logic of the system, including auction rules, bid validation, auction state transitions, and payment eligibility checks.
4. **Data Layer:** Manages persistent storage for users, auctions, catalogue items, bids, and payment records through dedicated databases accessed by the corresponding services.

This architecture ensures **high cohesion within components and low coupling between layers**, enabling clear separation of concerns and simplifying future maintenance and extension.

UML Component Diagram:



Services Table:

Service Name	Description	Exposed Interface Names	Interface Description
S1: Gateway (Façade)	Single entry point for the Web UI. Orchestrates UC1-UC7 workflows and enforces “one selected item per login session.”	S1:I1	S1:I1 Gateway Façade: UI-facing interface for sign-up/sign-in, catalogue search/selection, bidding, auction-ended/pay-now, payment submission, and receipt retrieval.
S2: IAM Service	Handles user registration, authentication, and password reset. Stores user profile and shipping address in User DB.	S2:I1	S2:I1 IAM API: Create user, authenticate user, reset password, and provide user shipping address.
S3: Catalogue Service	Manages auction item listings and item metadata (description, shipping cost, optional expedited shipping cost, shipping days). Supports keyword search and	S3:I1	Item metadata, search, visibility, shipping info, auction configuration (starting price, duration), and item lifecycle visibility (active/inactive)

	item retrieval.		
S4: Auction Service	Manages forward auction logic: current highest bid and bidder, bid validation (integer, strictly increasing), auction timer, and winner determination. Stores auction data in Auction DB.	S4:I1	S4:I1 Auction API: Bidding logic, bid validation, bid history, auction timing, winner determination, auction state
S5: Payment Service	Validates required payment fields and records payment outcome. Produces receipt details after successful payment. Stores payment records in Payment DB. The Payment Service assumes the user has already been authorized as the auction winner and does not perform winner validation.	S5:I1	S5:I1 Payment API: Submit payment and retrieve receipt data.
S6: Advanced Feature Service (Milestone 3 - Optional)	Optional Milestone 3 extension point. Provides AI-assisted / self-adaptive bidding support (UC8-x). Not required for Deliverable 1 implementation; included as a placeholder interface.	S6:I1	High-level API for starting an AI-bid assistant session and returning recommendations or auto-bid actions (Milestone 3 only).
S7: Image Storage Service	Handles image upload for seller item creation. Stores images and returns URLs/IDs that are saved with item metadata in the Catalogue DB.	S7:I1	Image upload + retrieval APIs used by Gateway during UC7.

Interfaces Table:

S1:I1 - Gateway Façade (Used by Web UI):

Interface Name	Operations	Operation Descriptions
S1:I1	SignUp(userProfile) : Result (returned by Web UI)	Creates user by forwarding data to IAM; requires username, password, first/last name, and shipping address.

	SignIn(username, password) : Session (used by Web UI)	Authenticates user via IAM; returns session/token for subsequent calls.
	ForgotPassword(resetInfo) : Result (used by Web UI)	Triggers password reset flow via IAM
	SearchItems(keyword) : ItemList (used by Web UI)	Returns list of matching item metadata from Catalogue; auction-specific data (current price, remaining time) is retrieved from Auction Service.
	SelectItem(itemId) : Result (used by Web UI)	Stores selected item for the current login session; enforces one item per session rule.
	GetBidPage(itemId) : BidPageDTO (used by Web UI)	Returns item details + current highest bid/bidder + remaining time for bidding page.
	PlaceBid(itemId, amount:int) : BidStateDTO (used by Web UI)	Submits bid to Auction service; validates integer and strictly increasing rule; returns updated state.
	GetAuctionEnded(itemId) : AuctionEndedDTO (used by Web UI)	Returns auction-ended view info and winner status; includes expedited shipping option from Catalogue.
	PayNow(itemId, expedited:boolean) : Result (used by Web UI)	Verifies winner eligibility and authorizes access to the payment flow.
	GetPaymentPage(itemId) : PaymentPageDTO (used by Web UI)	Loads payment page with user shipping address (IAM) + winning price (Auction) + shipping costs (Catalogue).
	SubmitPayment(itemId, paymentDetails) : ReceiptRef (used by Web UI)	Validates all payment fields are present; total = item price + shipping (+ expedited if selected).
	GetReceipt(receiptRef) : ReceiptDTO (used by Web UI)	Returns receipt info including total paid and “ships in xxx days”

		shipment text.
	UploadItem(itemInfo) : Result (used by Web UI / Seller)	Creates an auction item via Catalogue with duration, starting bid, and forward auction type.

S2:I1 - IAM API (Used by Gateway):

Interface Name	Operations	Operation Descriptions
S2:I1	CreateUser(userProfile) : Result (used by S1)	Stores user credentials and shipping address in User DB.
	Authenticate(username, password) : AuthResult (used by S1)	Verifies credentials for sign-in.
	ResetPassword(resetInfo) : Result (used by S1)	Updates the password for the user.
	GetShippingAddress(userId) : AddressDTO (used by S1)	Returns user address used on the payment page.

S3:I1 - Catalogue API (Used by Gateway):

Interface Name	Operations	Operation Descriptions
S3:I1	Search(keyword) : ItemList (used by S1)	Returns items matching keyword search.
	GetItemDetails(itemId) : ItemDTO (used by S1) [DTO: Data Transfer Object]	Returns item description, shipping cost, expedite cost (if any), and shipping days.
	CreateItem(itemInfo) : Result (used by S1)	Adds a new forward-auction item (seller upload).

S4:I1 - Auction API (Used by Gateway):

Interface Name	Operations	Operation Descriptions
S4:I1	GetAuctionState(itemId) : BidStateDTO (used by S1)	Returns highest bid, highest bidder ID, and remaining time.

	PlaceBid(itemId, bidderId, amount:int) : BidStateDTO (used by S1)	Accepts bids only if integer and strictly greater than current highest bid.
	GetWinner(itemId) : WinnerDTO (used by S1)	After auction end, returns winning bidder and winning bid price.

S5:I1 - Payment API (Used by Gateway):

Interface Name	Operations	Operation Descriptions
S5:I1	SubmitPayment(paymentDTO) : ReceiptRef (used by S1)	Requires card number, name, expiry, security code; records payment status.
	GetReceipt(receiptRef) : ReceiptDTO (used by S1)	Returns total paid and receipt info for UC6.

S6:I1 - Advanced API (Milestone 3 Placeholder):

Interface Name	Operations	Operation Descriptions
S6:I1	startAdvancedSession(context): SessionRef	Starts UC8-x AI feature session (optional Milestone 3).
	recommendBid(sessionRef, constraints): BidAdviceDTO	Returns recommended bid / next action (UC8-x).
	autoBid(sessionRef, maxBid:int)	Optional auto-bid mode up to a user-defined max bid (UC8-x).

S7:I1 - Image Storage Service:

Interface Name	Operations	Operation Descriptions
S7:I1	uploadImages(itemId, files[]): List<ImageRef>	Uploads one or more images and returns stored references/URLs (UC7).
	getImage(imageId): Binary/URL	Retrieves an image for display (UC2/UC7).

7. Activities Plan, Product Backlog, and Sprint Backlog:

7.1 Activities Plan

Our team will follow an Agile/Scrum methodology, divided into two major Sprints (Milestones). Each sprint will last approximately 4 weeks.

- **Sprint 1 (Milestone 2):** Focuses on the "Back-End First" approach. The goal is to establish the core microservices (IAM, Catalogue, Auction, Payment), set up the SQLite databases, and implement the Gateway API. The UI will be minimal but functional to demonstrate the use cases.
- **Sprint 2 (Milestone 3):** Focuses on "Deployment & Advanced Features." The goal is to polish the UI, containerize the application using Docker, implement the AI-assisted bidding feature (UC8), and ensure real-time updates via WebSockets.

Sprint Backlog: Milestone 2 (Weeks 1-4)

Selected items from Product Backlog to be completed for the next deadline:

1. **Environment Setup:** Initialize Spring Boot projects for IAM, Catalogue, Auction, Gateway.
2. **Database Design:** Implement SQLite schemas for UserDB, CatalogueDB, AuctionDB, and PaymentDB.
3. **UC1 (Sign-Up/In):** Implement IAM Service and JWT Token generation.
4. **UC2 (Browse):** Implement Catalogue Service (Search/List items).
5. **UC3 (Bidding):** Implement Auction Service (Place Bid logic, validation).
6. **UC4 (Auction End):** Implement timer logic to mark auctions as ENDED.
7. **UC5 & UC6 (Payment/Receipt):** Implement Payment Service and basic receipt generation.
8. **UC7 (Seller Upload):** Allow item creation via API.
9. **Integration:** Connect Gateway to backend services using REST Template/Feign.

Sprint Backlog: Milestone 3 (Weeks 5-8)

Selected items for the final release:

1. **Containerization:** Create Dockerfiles for all services and docker-compose.
2. **UC8 (Advanced AI):** Implement the Advanced Feature Service for bid recommendations.
3. **UI Polish:** Improve CSS/Styling for a cohesive user experience.
4. **Real-time Updates:** Implement WebSockets for live bid updates (push notifications).
5. **Testing:** Comprehensive integration testing and bug fixes.

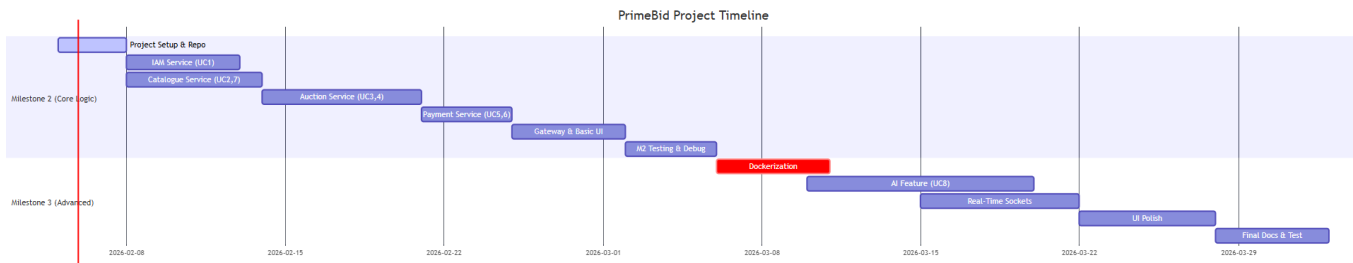
7.2 Product Backlog

ID	Description	Use Case	Priority	Milestone
1	As a user , I want to sign up and sign in to securely access the system	UC1	High	M2
2	As a bidder , I want to search and browse items so I can find products for auction	UC2	High	M2
3	As a bidder , I want to place increasing integer bids on items	UC3	High	M2
4	As a winner , I want to be notified when the auction ends so I can pay for my item	UC4	Medium	M2
5	As a winner , I want to be able to select expedited shipping	UC4	Medium	M2
6	As a winner , I want to submit my payment details to complete my purchase	UC5	High	M2
7	As a winner , I want a receipt and shipment timeline after payment	UC6	Medium	M2
8	As a seller , I want to upload item details and set auction durations	UC7	High	M2

10	As a user , I want to use AI-assisted bidding features (recommended bids and optional auto-bidding) to help me participate more effectively in auctions	UC8	Low	M3
11	As a bidder , I want my bid to be rejected if it is not an integer or not higher than the current bid	UC3	High	M2
12	As the system , I want to enforce that only one item can be selected per login session	UC2	High	M2
13	As the system , I want to automatically end auctions when their duration expires	UC4	High	M2
14	As the system , I want to determine whether an auction is sold or unsold when it ends	UC4	Medium	M2
15	As the system , I want to remove unsold or completed items from the active catalogue	UC4	Medium	M2
16	As the system , I want to restrict payment access to the winning bidder only	UC5	High	M2

17	As a user , I want clear error messages when an action is invalid or not permitted	UC1-UC6	Medium	M2
18	As the system , I want to securely store and manage user sessions after login	UC1	High	M2
19	As a seller , I want to upload images for auction items	UC7	Medium	M2
20	As the system , I want to persist all auction, bid, and payment data reliably	All	High	M2
21	As a developer , I want to define and execute test cases to validate system behavior (TDD)	All	High	M2
22	As a user , I want the system to log auction and payment outcomes for audit purposes	UC4 - UC6	Low	M2
23	As a user , I want the system to support AI-assisted bidding recommendations	UC8	Low	M3

7.3 GANTT Diagram



7.4 Group Meeting Logs

Meeting Date	Jan 23, 2026
Attendance	Nathan Binu Edappilly, Umer Shaikh, Imaduddin Ahmed, Pawanjot Randhawa (All Present)
Topics Discussed	1. Review of Project Description and "PrimeBid" requirements. 2. Analysis of the 7 Use Cases provided in the instructions. 3. Discussion on High-Level Architecture (Microservices vs. Layered/Monolithic). 4. Selection of technologies for the implementation phase.
Decisions Made	Architecture: Decided to use a Layered Architecture for Deliverable 1 to simplify the initial design, while keeping interfaces loose for a potential Microservices migration in Milestone 2. Use Case Split: Agreed to map UC1, UC2, UC3 for Sequence Diagrams and UC4, UC5, UC6, UC7 for Activity Diagrams to ensure no overlap. Tools: Selected Draw.io for UML modeling and Google Docs for collaboration.
Task Assignments	All Members: Read the Marking Schema and Deliverable Instructions in detail. Nathan: Initialize the Design Document structure and draft the Introduction/Overview. Umer: Begin drafting initial sketches for Sequence Diagrams (UC 1-3).

	Imaduddin: Research Controller/Service/Repository patterns for the Component Diagram.
--	--

Meeting Date	Jan 30, 2026
Attendance	Nathan Binu Edappilly, Umer Shaikh, Imaduddin Ahmed, Pawanjot Randhawa (All Present)
Topics Discussed	1. Review of initial Sequence and Activity diagram drafts. 2. Defining the API Interfaces (REST endpoints) for the Architecture section. 3. Planning the Product Backlog and GANTT chart timeline. 4. TDD Strategy: Brainstorming edge cases for the required 16 tests.
Decisions Made	Interface Definition: Finalized the "Auction Service" and "Catalogue Service" split. Agreed that the Gateway will handle session management. TDD Rule: Confirmed that each member must write exactly 4 test cases to meet the assignment requirement. Workload Division: Finalized the specific section ownership for the final report compilation.
Task Assignments	Umer: Finalize all Sequence and Activity Diagrams (clean up notation). Nathan: Complete the Architecture Section (Component Diagram + Interface Tables). Pawanjot: Create the Project Plan (GANTT/Backlog) and compile the final TDD table. Imaduddin: Integrate all sections, write the AI Acknowledgment, and perform final proofreading.

8. Test Driven Development:

TestID	TDD-TC-01
Category	Authentication / User Registration
Requirements Coverage	UC1.1-SignUp-Success
Initial Condition	System running; User DB available; username not already used.
Procedure	1. User opens the Sign-Up page. 2. User enters username, password, first name, last name, and shipping address. 3. User clicks Submit.
Expected Outcome	Account is created; user record stored in User DB; success message displayed; user can proceed to Sign-In.
Notes	Passwords should be stored securely (hashed), not plain text.

TestID	TDD-TC-02
Category	Authentication / User Registration
Requirements Coverage	UC1.1-SignUp-Validation-RequiredFields
Initial Condition	System running; User DB available.
Procedure	1. User opens Sign-Up page. 2. User leaves one required field blank (e.g., postal code). 3. User clicks Submit.
Expected Outcome	Validation error shown; no user record created.
Notes	Verify server-side validation, not UI-only.

TestID	TDD-TC-03
Category	Authentication / User Registration
Requirements Coverage	UC1.1-SignUp-Username-Unique

Initial Condition	System running; User DB contains username “nathan123”.
Procedure	1. User opens Sign-Up page. 2. User enters username “nathan123” with valid remaining details. 3. User clicks Submit.
Expected Outcome	Error message “username already exists”; no duplicate user created.
Notes	Ensure uniqueness enforced at DB level.

TestID	TDD-TC-04
Category	Authentication / Sign-In
Requirements Coverage	UC1.2-SignIn-Success
Initial Condition	System running; valid user exists in User DB.
Procedure	1. User opens Sign-In page. 2. User enters valid username and password. 3. User clicks Sign In.
Expected Outcome	User is authenticated; session established; user redirected to Catalogue page.
Notes	Session/token should be created and stored.

TestID	TDD-TC-05
Category	Authentication / Sign-In
Requirements Coverage	UC1.2-SignIn-InvalidCredentials
Initial Condition	System running; user exists in User DB.
Procedure	1. User opens Sign-In page. 2. User enters correct username but wrong password.

	3. User clicks Sign In .
Expected Outcome	“Invalid credentials” message displayed; no session created.
Notes	Avoiding leaks to which part was wrong.

TestID	TDD-TC-06
Category	Authentication / Forgot Password
Requirements Coverage	UC1.2-ForgotPassword-ResetSuccess
Initial Condition	System running; user exists; password reset enabled.
Procedure	1. User clicks Forgot Password. 2. User provides required reset input (e.g., username / token flow). 3. The user enters a new password and submits a reset. 4. The user signs in using the new password.
Expected Outcome	Password updated in User DB; user can sign in with new password.
Notes	Ensure old password no longer works.

TestID	TDD-TC-07
Category	Catalogue / Search
Requirements Coverage	UC2.1-ItemSearch-Keyword
Initial Condition	System running; Catalogue DB contains items with keywords (e.g., “watch”, “vintage”).
Procedure	1. User enters keyword in search bar. 2. User clicks Search
Expected Outcome	Matching items returned and displayed.
Notes	Search should use keywords stored in Catalogue DB.

TestID	TDD-TC-08
Category	Catalogue / Display Auctioned Items
Requirements Coverage	UC2.2-DisplayAuctionedItems-Fields
Initial Condition	System running; at least one auctioned item exists.
Procedure	1. User browses catalogue results. 2. User views item listings.
Expected Outcome	Each displayed item shows: full name, current bidding price, auction type (Forward), and remaining time (if applicable).
Notes	Forward auction type should be explicit.

TestID	TDD-TC-09
Category	Session / Item Selection
Requirements Coverage	UC2.3-ItemSelection-OnePerSession
Initial Condition	System running; user is logged in; multiple items displayed.
Procedure	1. User selects Item A using radio button. 2. User attempts to select Item B in same login session.
Expected Outcome	System enforces only one selected item per login session (either prevents second selection or replaces selection in a controlled way).
Notes	Align behavior with your Gateway rule; document which approach you implement.

TestID	TDD-TC-10
--------	-----------

Category	Auction / Place Bid
Requirements Coverage	UC3-Bid-ValidIncreasingInteger
Initial Condition	System running; auction active; current highest bid = 25.
Procedure	1. User opens selected item bidding page. 2. User enters bid = 30 (integer, higher than 25). 3. User clicks BID.
Expected Outcome	Bid accepted; Auction DB updated with new highest bid and highest bidder; UI refresh shows updated price.
Notes	Must be strictly higher, and integer only.

TestID	TDD-TC-11
Category	Auction / Bid Validation
Requirements Coverage	UC3-Bid-RejectNotIncreasing
Initial Condition	System running; user is logged in; an auction is ACTIVE for selected Item X; current highest bid = 25 ; user has selected Item X in this login session.
Procedure	1. User opens selected item bidding page for Item X. 2. User enters bid = 25 (equal to current highest). 3. User clicks BID. 4. User enters bid = 20 (lower than current highest). 5. User clicks BID. 6. User enters bid = 25.5 (non-integer). 7. User clicks BID.
Expected Outcome	• Bid = 25 is rejected with an error indicating the bid must be strictly higher than the current highest bid. • Bid = 20 is rejected with an error indicating the bid must be strictly higher than the current highest bid. • Bid = 25.5 is rejected with an error indicating bids must be integers only.

	• No changes occur in Auction DB: highest bid remains 25, highest bidder unchanged, and no new bid-history record is inserted for rejected bids.
Notes	Enforce server-side bid validation to prevent users bypassing rules via client-side manipulation. The auction state remains unchanged for invalid bids.

TestID	TDD-TC-12
Category	Auction / State Management
Requirements Coverage	UC4-AuctionEnded-UnsoldItem
Initial Condition	Auction for item X has N time left; No bid have been placed; N = 1 second
Procedure	1. Auction ends after N amount of time 2. Auction service trigger getWinner logic 3. Observe status of item X in Auction and catalogue database
Expected Outcome	In auction database auction should be marked as ENDED and item should be marked as UNSOLD; In catalogue database item should be removed;
Notes	Unsold items should be removed to avoid cluttering the catalogue database; Auction details can stay as the auction owner should be able to see the status of the auction

TestID	TDD-TC-13
Category	Payment / Authorization
Requirements Coverage	UC4-PaymentAccess-WinnersOnly
Initial Condition	Auction ended; User is logged in; User did not win the auction
Procedure	1. User click on the item page for the completed auction 2. User clicks "Pay Now"
Expected Outcome	UI displays a failure notice indicating that only the winning user is authorized to proceed to payment.
Notes	Gateway has to verify the user ID against the winner ID stored in

	the database.
--	---------------

TestID	TDD-TC-14
Category	Payment / Shipping Logic
Requirements Coverage	UC4-ExpeditedShipping-Calculation
Initial Condition	User if logged in; User has won the auction; User has clicked pay now and has been redirected to the payment screen; Winning bid =\$100, Expedited Shipping = \$25
Procedure	1. User selects "expedite shipping" option 2. User can observe updated total amount
Expected Outcome	UI should display a total of \$125
Notes	Expedited cost should be pulled from the database for each specific item.

TestID	TDD-TC-15
Category	Payment / Input Validation
Requirements Coverage	UC5-Payment-FieldValidation
Initial Condition	User is logged in; User has won auction and has navigated to payment details page
Procedure	1. User enters Name, Card Number, Expiration 2. User does not fill out security code field 3. User clicks "Submit"
Expected Outcome	The system displays a validation error indicating that all fields must be filled in.
Notes	System should also check proper formatting for each field, so not character in card number; Transaction should not be recorded is validation error occurs

TestID	TDD-TC-16
Category	Receipt / Shipping Details
Requirements Coverage	UC6-ReceiptPage-ShowTotalsAndShipping
Initial Condition	System running; user is logged in; user is the winning bidder for Item X; payment for Item X is COMPLETED in Payment DB; Catalogue DB contains Item X shipping timeline text (e.g., “Ships in 5 days”).
Procedure	1. After payment success, user is redirected to the Receipt & Shipping Details page (UC6). 2. System retrieves transaction details (transactionId, winningBid, shipping cost, total paid). 3. System retrieves item shipping info from Catalogue DB (e.g., “Ships in 5 days”). 4. User views the page.
Expected Outcome	• Receipt page displays: transaction ID, item name, winning price, shipping cost, and total paid. • Shipping section displays a timeline statement such as: “Item will be shipped in X days” (text is acceptable as per instructions). • The page provides a navigation option such as Back to Main / Return Home.
Notes	Shipping timelines are displayed as descriptive text rather than computed calendar dates, in accordance with the project requirements. The receipt page is read-only and reflects only finalized transaction and shipment information.

TestID	TDD-TC-17
Category	Seller / Auction Item Upload
Requirements Coverage	UC7-CreateAuctionItem-Success
Initial Condition	System running; seller user is logged in; Catalogue DB and Auction DB available; image upload mechanism (Most

	possibly) available.
Procedure	1. Seller clicks Create Auction. 2. Seller enters valid item details: name, description, keywords, starting price, auction duration, shipping info, and auction type = Forward. 3. Seller uploads at least one image (if your implementation supports images). 4. Seller clicks Submit/Create.
Expected Outcome	• Item is created in Catalogue DB with status ACTIVE and searchable keywords stored. • A corresponding auction record is created in Auction DB with start time, end time, starting bid, and status ACTIVE. • UI displays a success message with a generated ItemID.
Notes	This test confirms that a seller can successfully create a new auction and that the item becomes visible for browsing and bidding. Image upload can be treated as optional or mocked if not fully implemented in this milestone.

TestID	TDD-TC-18
Category	Seller / Auction Item Upload Validation
Requirements Coverage	UC7-CreateAuctionItem-ValidationFailure
Initial Condition	System running; seller user is logged in; Catalogue DB and Auction DB available.
Procedure	1. Seller clicks Create Auction. 2. Seller enters invalid values (choose at least one): • starting price = 0 or negative

	• auction duration outside allowed range (e.g., 0 minutes or > 30 days) • missing required fields (e.g., name or keywords empty) 3. Seller clicks Submit/Create.
Expected Outcome	• Validation error is displayed describing the invalid field(s). • No item record is created in Catalogue DB and no auction record is created in Auction DB.
Notes	Enforce server-side validation for business rules, not UI-only.

9. Acknowledgment (AI Usage):

We utilized Generative AI tools (specifically Large Language Models) to assist in the drafting and organization of this design document. The AI was used primarily as a productivity tool to streamline the documentation process and provide an organizational understanding of the system's execution. Specific areas where AI assistance was utilized include:

- **Test Case Generation:** We used AI to brainstorm potential edge cases and generate the initial list of Test Driven Development (TDD) scenarios (Section 8) to ensure comprehensive requirements coverage.
- **Implementation Strategy & Logic Mapping:** The AI was used to help us gain an organizational understanding of how to plan out the implementation of the project. This included identifying logical dependencies between system components to ensure our diagrams accurately reflected a coherent workflow.
- **Project Planning & Scheduling:** AI was used to help structure the Activities Plan and organize the sequential data required for the GANTT chart (Section 7), ensuring the development timeline was realistic and logically ordered.

Critical Evaluation of Generative AI

As required by the project specifications, the team conducted a critique of the GenAI tools used based on our experience during this deliverable:

Strengths:

- **Rapid Prototyping of Documentation:** AI significantly reduced the time required to format complex tables (such as the TDD and Service Interface tables) and helped in drafting the initial structure of the document.

- **Edge Case Discovery:** The AI was effective at suggesting "negative" test scenarios (e.g., bidding after an auction has ended or entering non-integer values) that ensured our TDD section was robust.
- **Synthesized Planning:** It helped translate our high-level deadlines into a structured sequence of tasks suitable for a GANTT diagram.

Weaknesses:

- **Inconsistent UML Notation:** We observed that the AI frequently suggested diagram logic or labels that did not strictly adhere to standard UML notation or the specific architectural constraints of our system. For instance, initial suggestions for component connections often ignored the requirement for a Gateway/Facade.
- **Technical Hallucinations:** On several occasions, the AI proposed REST endpoints or data types that were inconsistent with our established "PrimeBid" database schema, requiring manual correction to ensure consistency across the Sequence and Architecture diagrams.
- **Generic Outputs:** The initial AI-generated descriptions were often too general; the team had to perform significant manual refining to ground the content in the specific "Forward Auctioning" business rules required for EECS 4413.

Disclaimer: All content generated by AI was critically reviewed, verified, and refined by the team members to ensure it accurately reflects our architectural decisions and meets the specific constraints of the project requirements.

EECS 4413 Deliverable 2 Report

Author(s)
Nathan Binu Edappilly 219317965
Umer Shaikh 218 931 790
Imaduddin Ahmed 219 569 029
Pawanjot Randhawa 219 732 718

Table of Contents - Milestone 2

- Abstract
- 10. Scope and Milestone 2 Constraints
 - 10.1 Scope
 - 10.2 Explicit Milestone Constraint
- 11. Deviations and Updates from Milestone 1
 - 11.1 Architecture and Service Boundaries
 - 11.2 Validation and Robustness Updates
 - 11.3 Consistency and Hypermedia
 - 11.4 Testing Strategy Evolution
 - 11.5 Clarification of "Real-Time" Scope and Behavior
 - 11.6 Deliverable 1 to Deliverable 2 Deviation Map
- 12. Alternative Architecture Comparison (Expanded)
- 13. Coupling/Cohesion Justification with Concrete Evidence
 - 13.1 Low Coupling Evidence
 - 13.2 High Cohesion Evidence
 - 13.3 Consistency Check (Architecture Narrative vs Diagram)
- 14. Design Patterns Used (4)
 - 14.1 Facade Pattern (Gateway)
 - 14.2 Repository Pattern (Persistence Boundary)
 - 14.3 Strategy Pattern (Bid Validation Rules)
 - 14.4 Observer Pattern (Auction End Side Effects)
- 15. Design Patterns Considered but Not Used (2)

- 15.1 Abstract Factory (Considered)
- 15.2 Manual Singleton (Considered)
- 16. Technology Choices and Justification
- 17. Use-Case and Test Completeness Matrix
- 18. Diagram Inventory
- 19. Diagram Walkthrough
- 20. AI Usage Acknowledgement (Advantages and Disadvantages)
 - 20.1 Scope of AI Use
 - 20.2 Human Oversight and Validation
 - 20.3 Advantages
 - 20.4 Limitations / Risks
 - 20.5 Mitigation Measures
- 21. Team Contribution
- 22. Team Meeting Logs
- 23. Mark Alignment
 - 23.1 Implementation Mark (/80) Alignment
 - 23.2 Report Content Mark (/15) Alignment
 - 23.3 Report Writing Quality Mark (/5) Controls:
- 24. References
 - 24.1 Academic & Course Materials
 - 24.2 Technical Frameworks & Security
 - 24.3 Design Patterns & Modeling
 - 24.4 Project Artifacts & Scripts

Abstract

This addendum documents the Deliverable 2 backend implementation of UC1-UC7, associated testing artifacts, and architecture/design updates relative to Deliverable 1. The implemented solution adopts a gateway-facade with service decomposition across IAM, Catalogue, Auction, and Payment, with service-owned persistence per bounded context. The document further explains design-pattern application, architectural tradeoffs, and implementation-consistency decisions (including scope boundaries for real-time behavior), and provides traceable evidence for scripts and tests used in evaluation.

10. Scope and Milestone 2 Constraints

10.1 Scope

- Implementing backend UC1-UC7 with RESTful endpoints.
- Providing executable scripts for primary-use-case flow and robustness testing.
- Providing installation, execution, and testing instructions, together with report-level design
- justification.

10.2 Explicit Milestone Constraint

- Real-time client notification when bidding ends is not required in Deliverable 2 and is intentionally out of scope.

11. Deviations and Updates from Milestone 1

11.1 Architecture and Service Boundaries

- Finalized five-service decomposition:
 - Gateway (facade/orchestration)
 - IAM (authentication/profile)

- Catalogue (item data/status)
- Auction (auction state/bidding/end transitions)
- Payment (payment/receipt)
- Introduced stricter orchestration checks in gateway flows (selection, payment, receipt authorization).

11.2 Validation and Robustness Updates

- Added/strengthened server-side validation for:
 - required UC7 fields (including keywords)
 - Bid correctness rules (integer/increasing/active auction)
 - Payment constraints and duplicate prevention safeguards
 - Reset-password identity checks and credential validation

11.3 Consistency and Hypermedia

- Gateway business-flow responses include _links to support discoverability and next-step navigation.

11.4 Testing Strategy Evolution

- Extended testing beyond endpoint smoke checks to include:
 - Module-level automated tests
 - Scripted end-to-end main flow
 - Robustness scripts for invalid/wrong inputs
 - Concurrency-focused regressions in auction/payment services

11.5 Clarification of "Real-Time" Scope and Behavior

Deliverable 2 does not implement push notifications or WebSockets for auction-end events. In this milestone, "real-time bidding" denotes immediate server-side validation and state transition per bid request, with clients obtaining updates through refresh/re-query behavior. Auction-end transitions are scheduler-driven and observable via query endpoints (e.g., */auction-result/{itemId}*), rather than push transport.

11.6 Deliverable 1 to Deliverable 2 Deviation Map

Design Aspect	Deliverable 1 Baseline	Deliverable 2 Implementation Update	Why the Change Matters
Service partitioning	High-level decomposition was proposed	Fully implemented gateway + 4 backend services with service-owned persistence	Makes bounded contexts explicit and testable per service
Use-case orchestration ownership	Gateway role described conceptually	Gateway now performs concrete UC flow orchestration and state checks	Enforces business preconditions consistently at one entry point
Data persistence	Separated was planned	IAM/Catalogue/Auction/Payment each use their own DB schema and repository layer	Reduces direct cross-domain coupling and shared-state risk
Validation strategy	General validation intent described	Strong input validation added across DTOs/controllers/services	Improves robustness scores for

		(including UC7 required fields and bid/payment checks)	wrong-input scenarios
Auction end behavior	End-of-auction flow modeled	Scheduler + observer callback implemented; result exposed via query endpoints	Implements required behavior while staying within D2 scope
Concurrency handling	Not deeply specified in D1	Concurrency safeguards added for bidding and duplicate-payment prevention	Prevents correctness regressions under concurrent requests
Test strategy	TDD intent and case ideas	Implemented module tests + robustness scripts + end-to-end scripts	Provides direct evidence for grading on coverage and resilience
Hypermedia consistency	REST/HATEOA S stated as principle	Gateway business-flow responses include _links with next-step navigation	Improves API discoverability and rubric alignment

Implementation evidence (illustrative):

- Gateway orchestration/controllers:
gateway/src/main/java/com/forwardauction/gateway/
- Auction scheduler/observer:
auction/src/main/java/com/forwardauction/auction/service/AuctionScheduler.java
- Strategy validators:
auction/src/main/java/com/forwardauction/auction/strategy/
- Repository layer usage: **/src/main/java/**/repo/*
- Robustness scripts: *scripts/robustness/test_robustness.sh*

12. Alternative Architecture Comparison (Expanded)

Option	Advantages	Tradeoffs / Why Not Chosen for D2
Monolith (single deployable)	Lowest operational complexity; easiest local debugging	Weak service boundary evidence, limited independent evolution, less alignment with milestone progression toward microservices/containerization
Gateway + service decomposition (chosen)	Clear bounded contexts, independent testability, explicit API contracts, strong fit for facade/orchestration use cases	More inter-service calls and coordination complexity
Fully distributed event-driven (async messaging first)	High decoupling and scalability patterns	Added infra/ops complexity unjustified for D2 timeline and requirements; no push notification requirement in D2

Selected rationale:

- The gateway plus service decomposition provides clear separation of concerns and rubric-aligned architectural evidence without introducing distributed-systems complexity beyond Deliverable 2 requirements.

13. Coupling/Cohesion Justification with Concrete Evidence

13.1 Low Coupling Evidence

- Service-owned databases reduce direct data coupling across domains.
- Cross-service integration is via HTTP APIs, not shared in-process domain objects.
- Gateway centralizes orchestration while backend services keep domain logic localized.

13.2 High Cohesion Evidence

- IAM: authentication/profile concerns only.
- Catalogue: item catalog state and search concerns only.
- Auction: bidding, expiry, and auction-result concerns only.
- Payment: payment persistence and retrieval concerns only.
- Gateway: session handling, orchestration, and HATEOAS assembly concerns.

13.3 Consistency Check (Architecture Narrative vs Diagram)

- This project uses a microservice-style decomposition with a gateway facade.
- The component diagram intentionally reflects this architecture (service split + service-owned datastores).
- The report explicitly states we did not implement a fully distributed asynchronous/event-bus architecture for D2, avoiding contradiction with the component diagram.

14. Design Patterns Used (4)

14.1 Facade Pattern (Gateway)

- Intent:
 - Provide a single, stable API entry point for all client interactions while hiding backend service topology.
- Applied in:
 - Gateway controllers under:
gateway/src/main/java/com/forwardauction/gateway/.
 - Examples: *AuthProxyController, CatalogueController, BidController, ItemOrchestratorController, PaymentOrchestratorController*
- Concrete behavior:
 - Aggregates data from multiple services for UC2, UC5, UC6.
 - Enforces cross-service preconditions (session validity, selected item, winner-only payment/receipt).
 - Adds HATEOAS links on business-flow responses.
- Tradeoff:
 - Centralized orchestration can grow complex; mitigated by splitting flows across focused controllers.

14.2 Repository Pattern (Persistence Boundary)

- Intent:
 - Isolate SQL/data-access logic from business logic for maintainability and testability.
- Applied in:
 - *iam/.../repo/UserRepository.java*
 - *catalogue/.../repo/ItemRepository.java*
 - *auction/.../repo/AuctionRepository.java* ,
 - *auction/.../repo/BidRepository.java*
 - *payment/.../repo/PaymentRepository.java*
- Concrete behavior:
 - Services call repository methods, not raw SQL directly.
 - Schema ownership is kept per service.
- Tradeoff:

- Slightly more boilerplate, but clearer separation and easier test targeting.

14.3 Strategy Pattern (Bid Validation Rules)

- Intent:
 - Make bid validation rules modular and replaceable without changing core bidding flow.
- Applied in:
 - *auction/.../strategy/BidValidationStrategy.java*
 - *auction/.../strategy/IntegerBidValidator.java*
 - *auction/.../strategy/IncreasingBidValidator.java*
- Concrete behavior:
 - *AuctionService.placeBid(...)* iterates through strategy validators before attempting update.
 - Keeps rule evolution localized (e.g., min increment rule can be added as a new strategy class).
- Tradeoff:
 - More classes than inline validation, but better extensibility and test focus.

14.4 Observer Pattern (Auction End Side Effects)

- Intent:
 - Decouple auction-end state transition from follow-up actions.
- Applied in:
 - *auction/.../observer/AuctionEndObserver.java*
 - *auction/.../observer/CatalogueNotifier.java*
 - Triggered by: *auction/.../service/AuctionScheduler.java*
- Concrete behavior:
 - The scheduler identifies expired auctions and emits onAuctionEnded(...) notifications.
 - Observers then update downstream catalogue status without embedding this concern into the core auction write path.
- Tradeoff:
 - Additional moving parts; however, this is justified by a cleaner separation of responsibilities.

15. Design Patterns Considered but Not Used (2)

15.1 Abstract Factory (Considered)

- Why it was considered:
 - Potential extension path for multiple auction families (forward, dutch, sealed-bid) with shared creation contracts.
- Why not used in D2:
 - Milestone scope only requires forward auction.
 - A factory hierarchy would add abstractions not exercised by current use cases.

- Future trigger to adopt:
 - If a new milestone introduces multiple auction families with differing creation/validation workflows.

15.2 Manual Singleton (Considered)

- Why it was considered:
 - Shared components such as connection/state managers often use singleton-style lifecycle.
- Why not used in D2:
 - Spring IoC already provides a singleton bean lifecycle where appropriate.
 - Manual singleton introduces tighter coupling and harder testing/mocking.
- Future trigger to adopt:
 - Only if a component must be managed outside Spring context (not the case in current architecture).

16. Technology Choices and Justification

Technology	Choice Rationale
Spring Boot	Fast REST development, strong validation/testing ecosystem, standard enterprise patterns.
Java 17	Stable LTS baseline with broad tooling support.
SQLite per service	Lightweight local persistence suitable for milestone scope and independent service ownership.
Spring JDBC (<i>JdbcTemplate</i>)	Explicit SQL control and low abstraction overhead for required schemas/use cases.
RestTemplate	Simple inter-service HTTP orchestration for current milestone requirements.
JUnit 5 + MockMvc	Service-level automated verification of success paths and robustness behavior.

17. Use-Case and Test Completeness Matrix

Use Case	Main Flow Evidence	Robustness / Wrong Input Evidence	Notes
UC1 Authentication	<i>scripts/main_flow/full_flow.sh</i>	<i>scripts/robustness/test_robustness.sh</i> , IAM tests	Signup/login/reset/me/logout covered.
UC2 Browse/Search/Select	<i>scripts/main_flow/full_flow.sh</i>	Gateway and catalogue controller tests + robustness script	Includes select authorization/state checks.
UC3 Place Bid	<i>scripts/main_flow/full_flow.sh</i>	Auction tests + robustness script	Includes invalid/low/non-increasing bid handling.
UC4 Auction End	<i>scripts/main_flow/full_flow.sh</i>	Auction service/controller tests	Scheduler-driven state transition validated.
UC5 Payment	<i>scripts/main_flow/full_flow.sh</i>	Payment tests + robustness script	Includes premature payment rejection and duplicate safeguards.
UC6 Receipt	<i>scripts/main_flow/full_flow.sh</i>	Gateway payment/receipt tests	Includes ownership authorization checks.

UC7 Create Item + Auction	<i>scripts/main_flow/full_flow.sh</i>	UC7 validation scripts/tests	Required-field and duration validation included.
---------------------------	---------------------------------------	------------------------------	--

*Automated test suites are located under each module at */src/test/java/.*

18. Diagram Inventory

Source files are included in the repository under *docs/diagrams/* within the repository:

Diagram Type	Use Case / Scope	File
Subsystem / component	Overall D2 architecture	<i>docs/diagrams/component_d2_as_built.png</i>
Class diagram	Auction subsystem	<i>docs/diagrams/class_auction_subsystem_d2.png</i>
Class diagram	Gateway orchestration subsystem	<i>docs/diagrams/class_gateway_subsystem_d2.png</i>
Sequence diagram	UC1 (signup/login)	<i>docs/diagrams/sequence_uc1_auth_d2.png</i>
Sequence diagram	UC2 (browse/search/select)	<i>docs/diagrams/sequence_uc2_browse_select_d2.png</i>
Sequence diagram	UC3 (place bid)	<i>docs/diagrams/sequence_uc3_bid_d2.png</i>
Sequence diagram	UC5/UC6 (payment + receipt)	<i>docs/diagrams/sequence_uc5_uc6_payment_receipt_d2.png</i>
Activity diagram	UC4 (auction end result)	<i>docs/diagrams/activity_uc4_auction_end_d2.png</i>

Activity diagram	UC5 (payment submit)	<i>docs/diagrams/activity_uc5_payment_submit_d2.png</i>
Activity diagram	UC6 (receipt)	<i>docs/diagrams/activity_uc6_receipt_d2.png</i>
Activity diagram	UC7 (create item + auction)	<i>docs/diagrams/activity_uc7_create_item_d2.png</i>

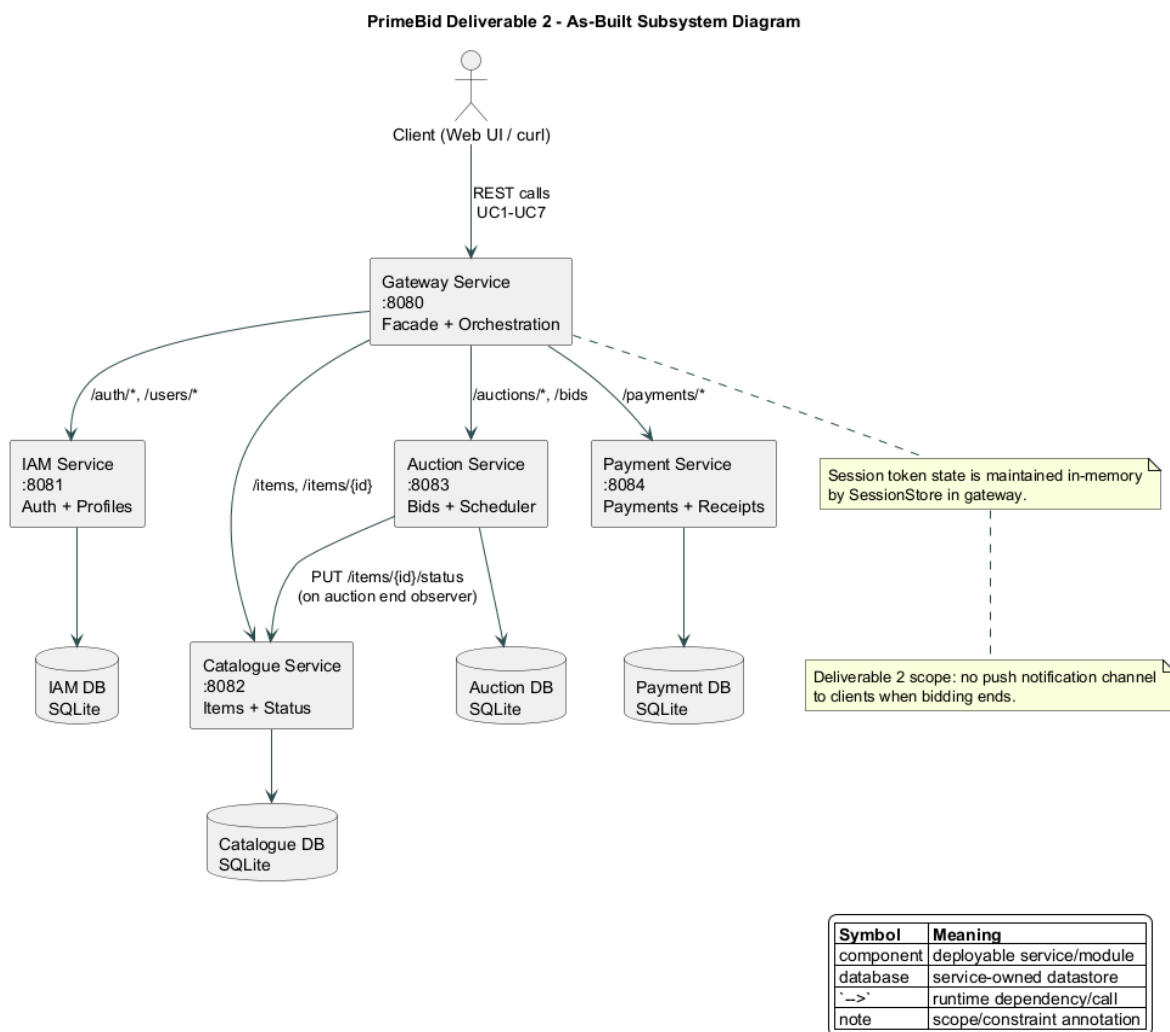
Export instructions are provided in:

- *docs/diagrams/README.md*

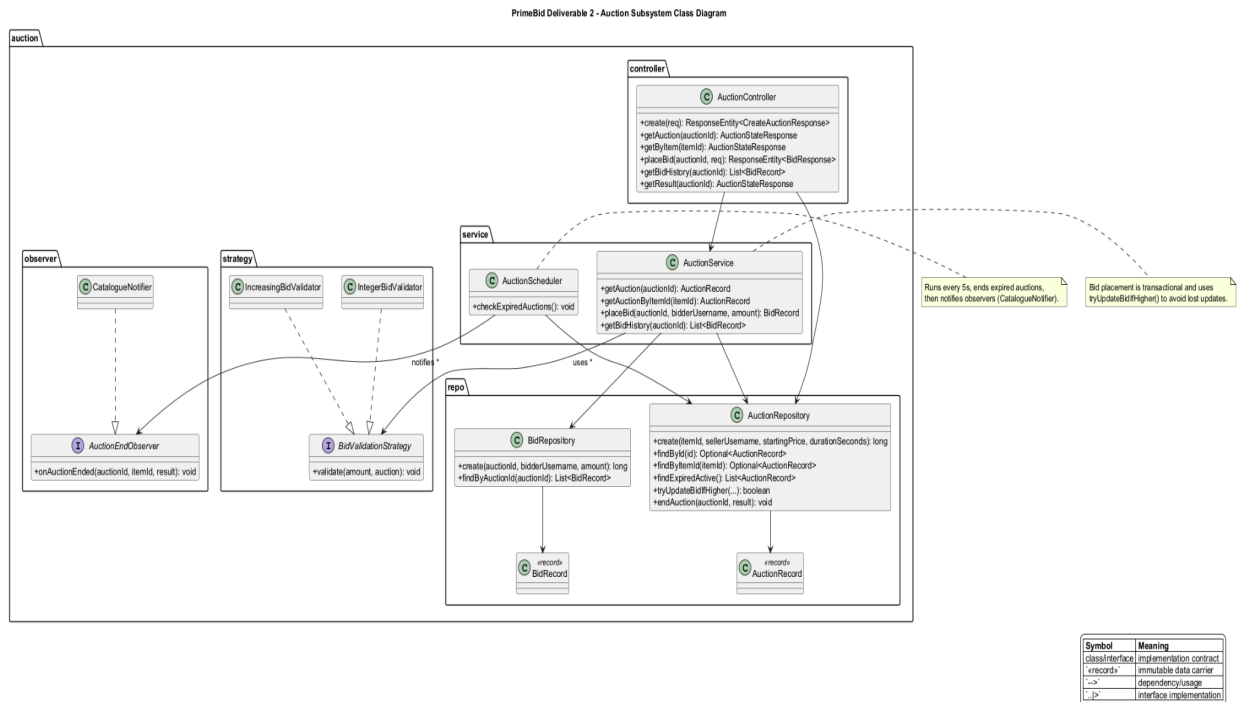
Diagram insertion checklist for the final PDF/report:

- Include legends where notation is non-obvious.
- Ensure diagram labels match implementation names used in code.
- Ensure fonts/arrows remain readable after PDF export or screenshot insertion.

19. Diagram Walkthrough



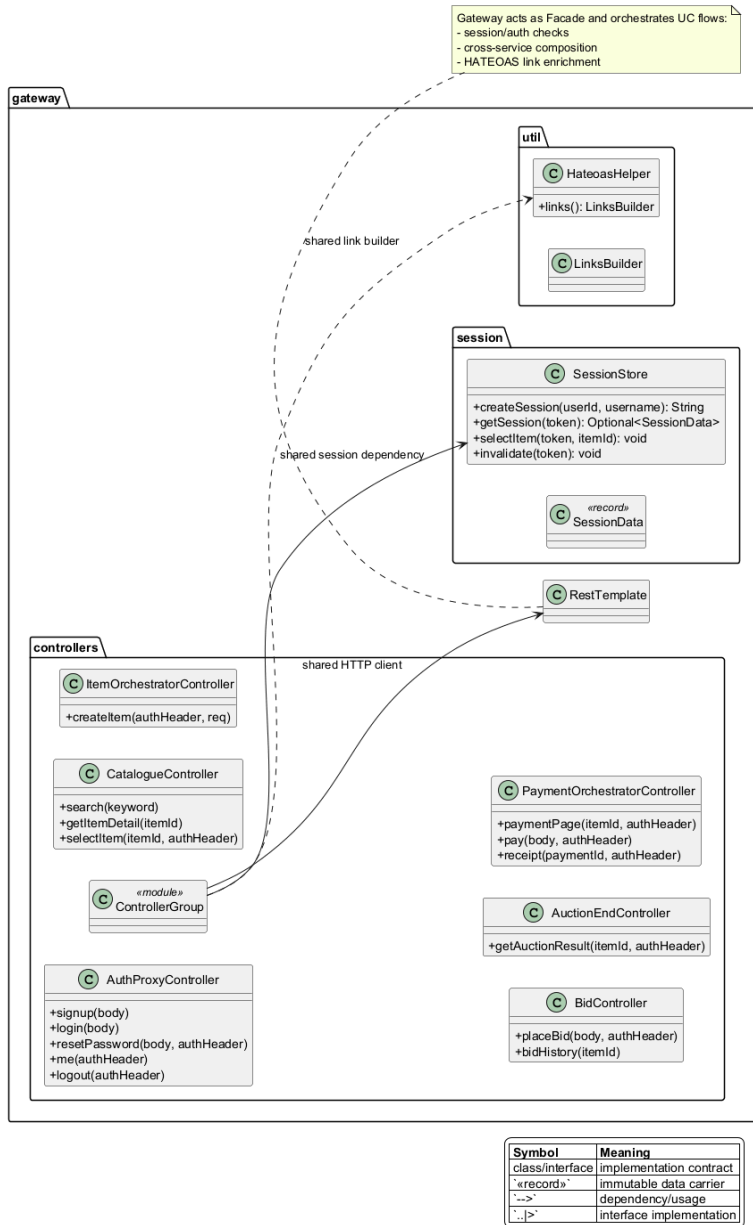
- **component_d2_as_built :**
 - Read top-down from client to gateway to backend services.
 - Note the explicit auction-to-catalogue observer update path and D2 constraint note (no client push notifications).



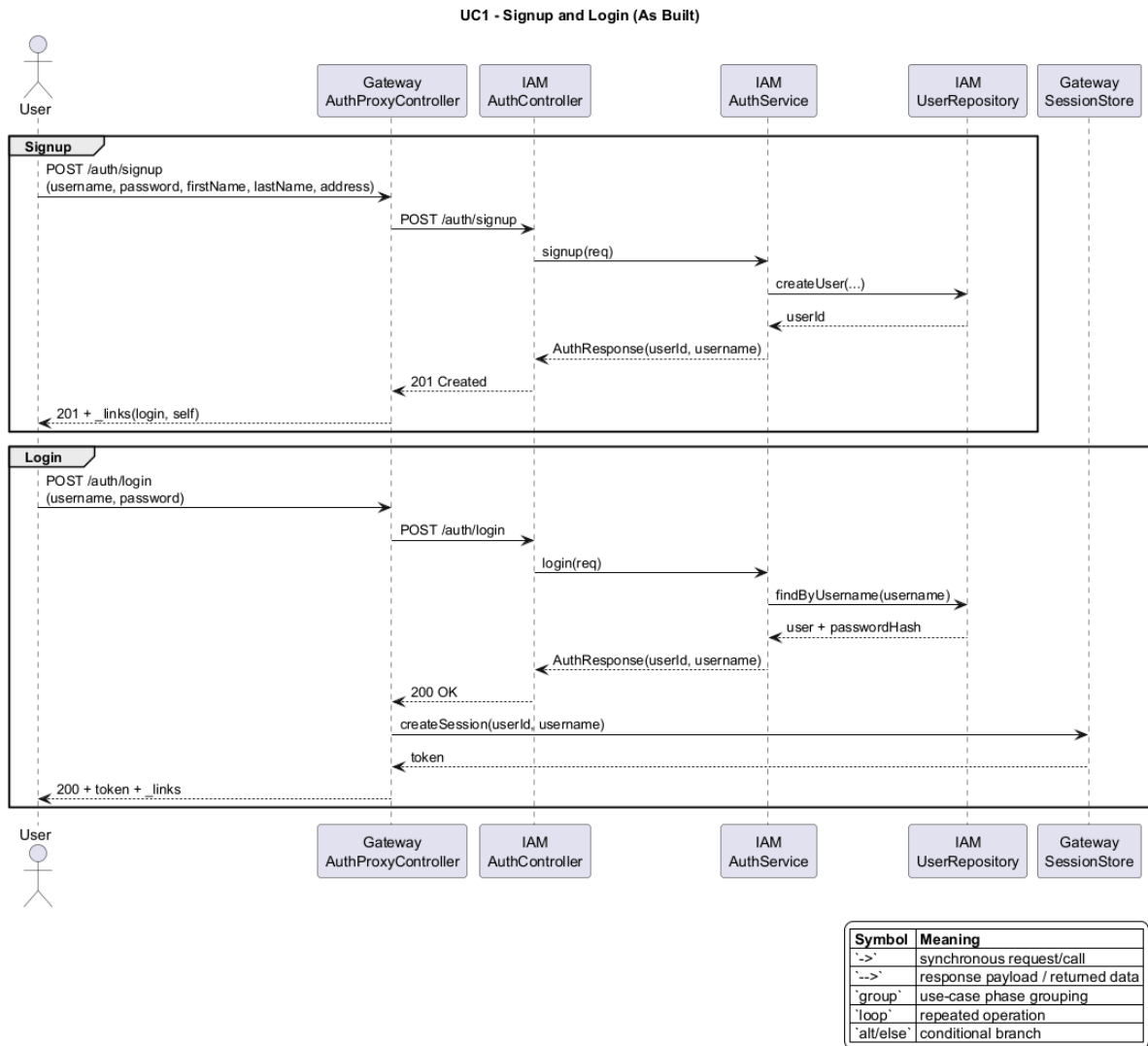
- **class_auction_subsystem_d2 :**

- Follow AuctionController -> AuctionService -> repositories
- Focus on validator strategy plug-in points and observer notification from scheduler.

PrimeBid Deliverable 2 - Gateway Orchestration Class Diagram

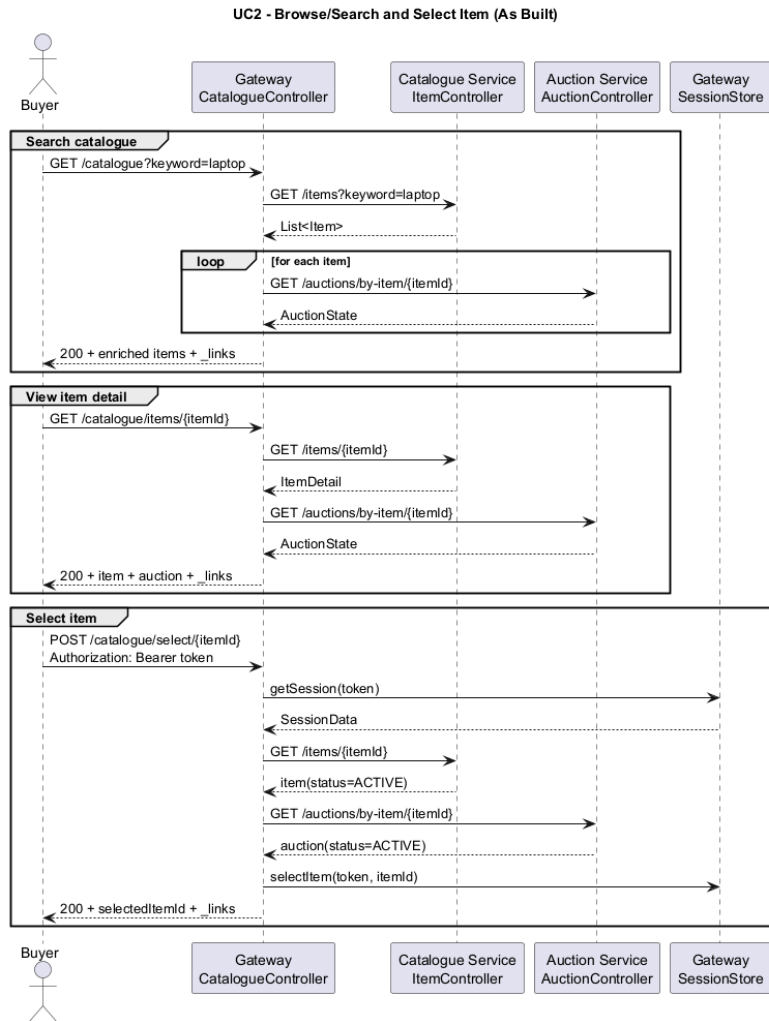


- **class_gateway_subsystem_d2 :**
 - Shows gateway facade/orchestration role and shared dependencies (SessionStore , RestTemplate , HateoasHelper).
 - Each controller maps to one UC flow boundary.



- **sequence_uc1_auth_d2 :**

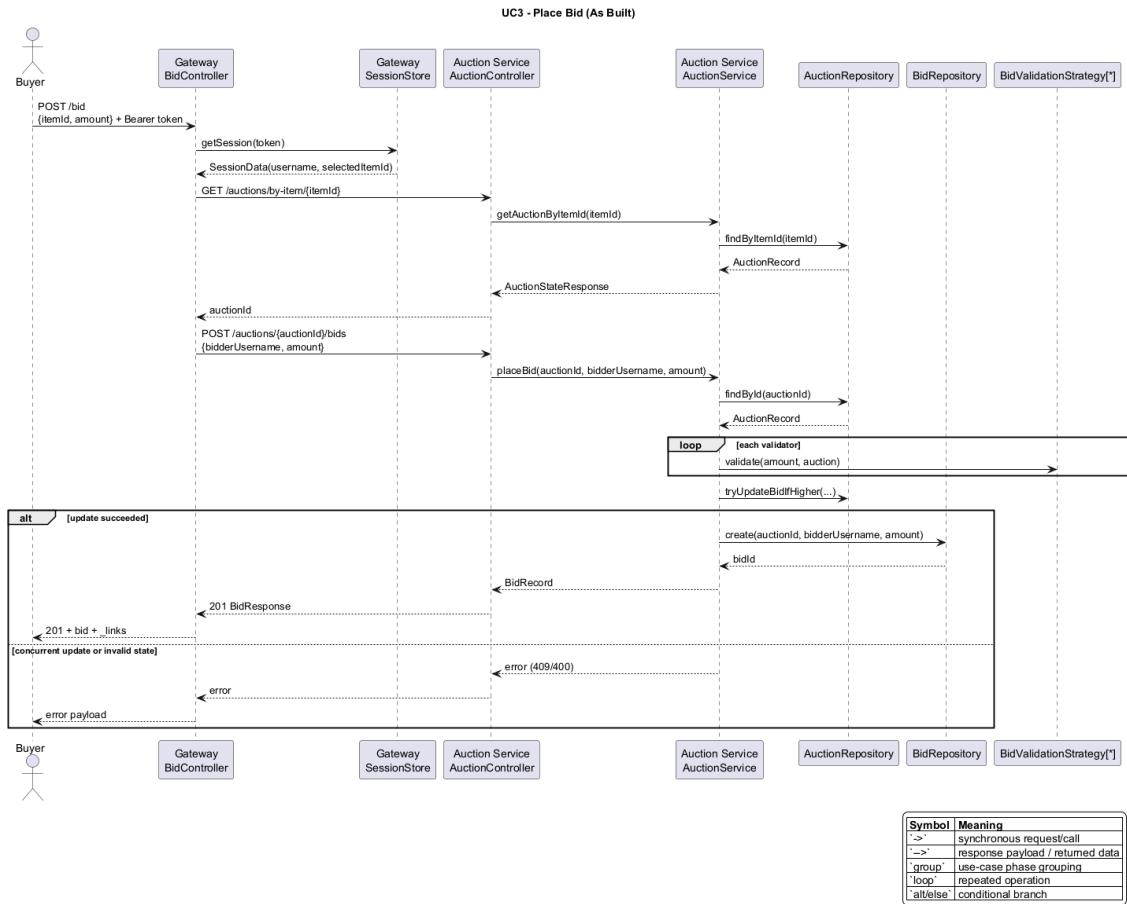
- The signup path ends at IAM user creation.
- Login path highlights session token creation in gateway.



Symbol	Meaning
'->'	synchronous request/call
'-->'	response payload / returned data
'group'	use-case phase grouping
'loop'	repeated operation
'alt/else'	conditional branch

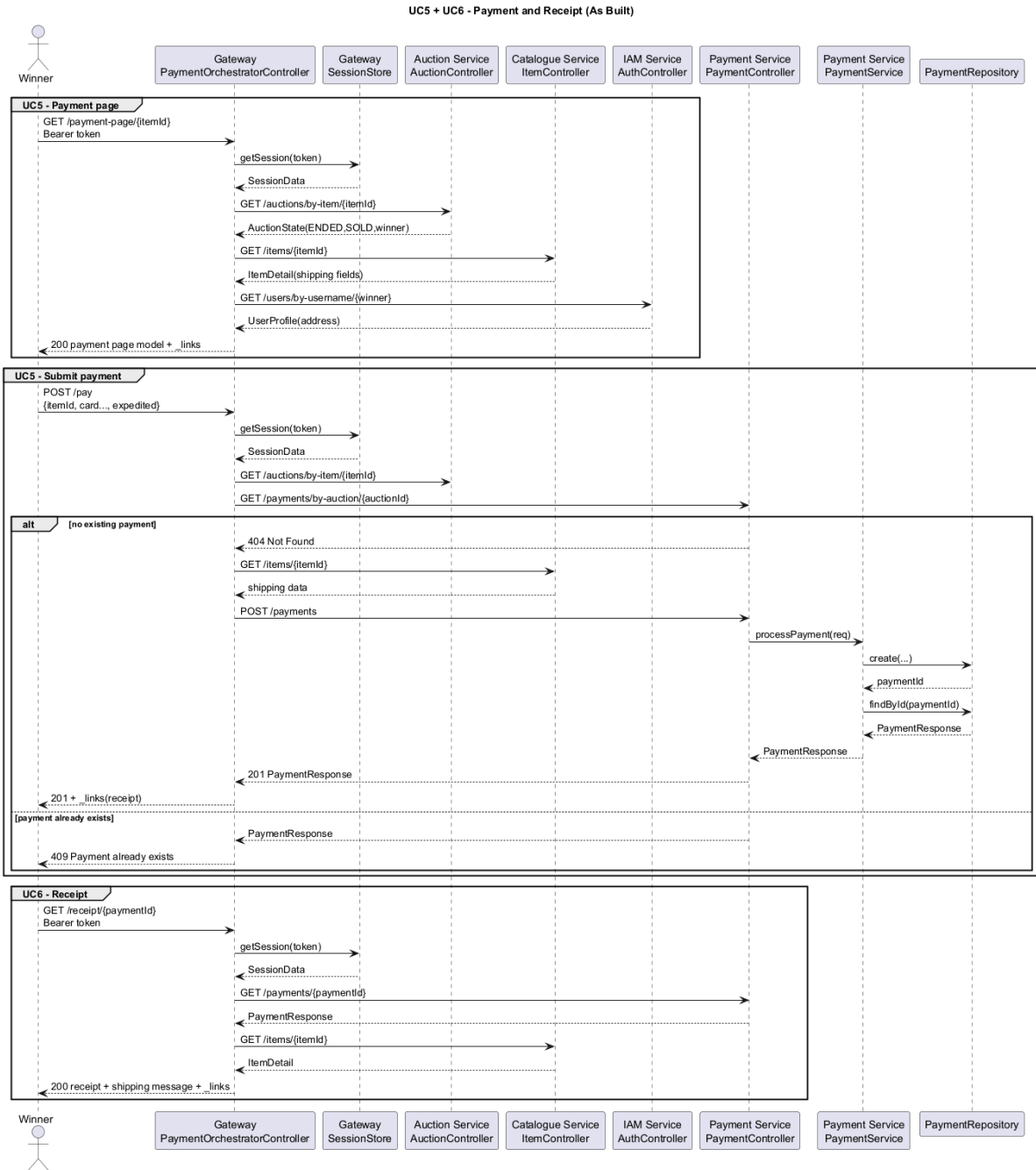
- **sequence_uc2_browse_select_d2 :**

- Search and detail flows enrich catalogue data with auction state.
- Selection flow shows auth/session check plus ACTIVE state checks before storing selected items.



- **sequence_uc3_bid_d2:**

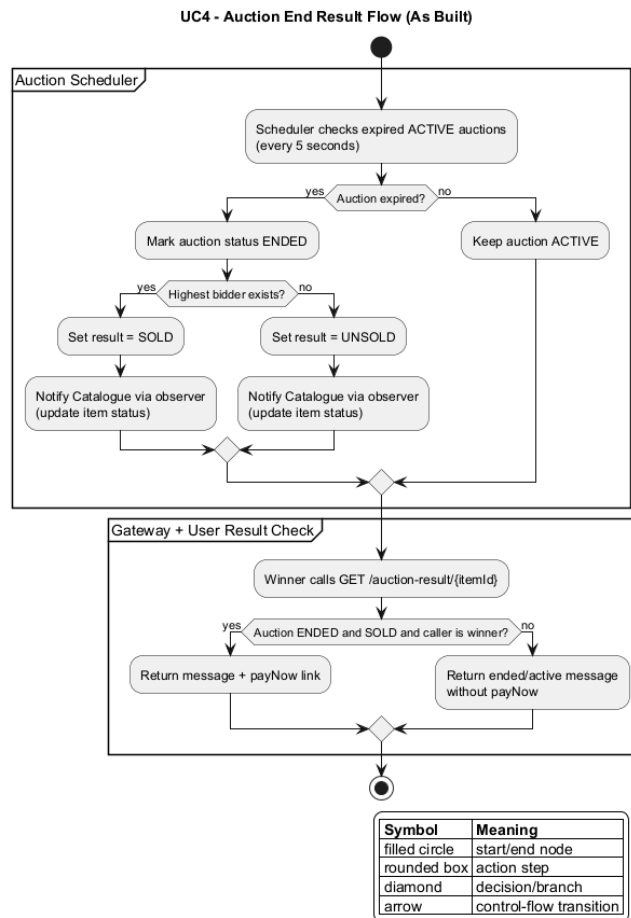
- Highlights selected-item/session guard, auction lookup, strategy validation, and concurrency-safe update (tryUpdateBidIfHigher).



Symbol	Meaning
'->'	synchronous request/call
'-->'	response payload / returned data
'group'	use-case phase grouping
'loop'	repeated operation
'alt/else'	conditional branch

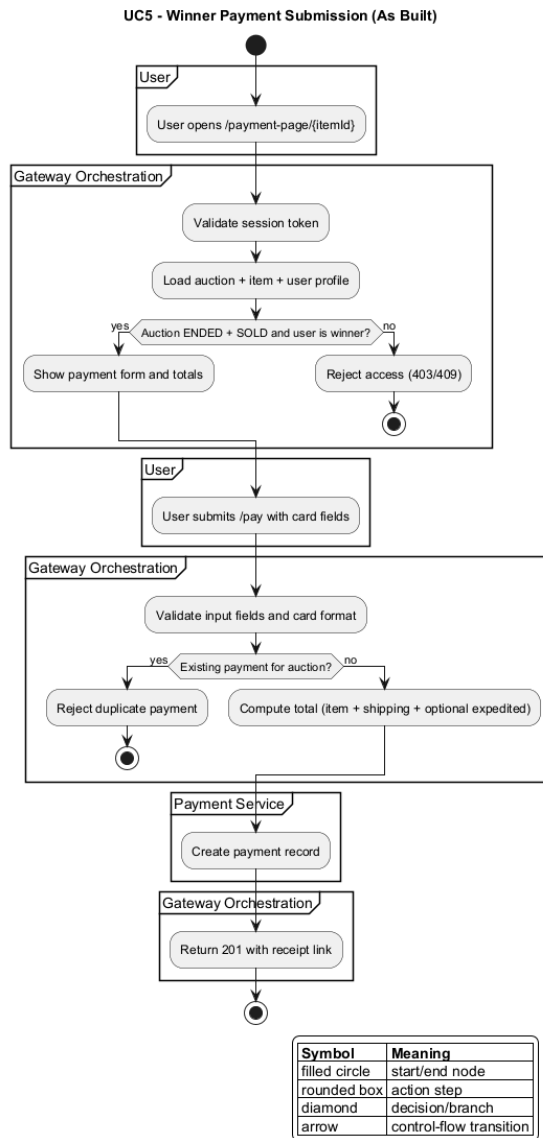
● **sequence_uc5_uc6_payment_receipt_d2 :**

- Payment page requires winner/ended/sold validation.
- Payment submission includes duplicate-payment pre-check and receipt ownership enforcement.



- **activity_uc4_auction_end_d2 :**

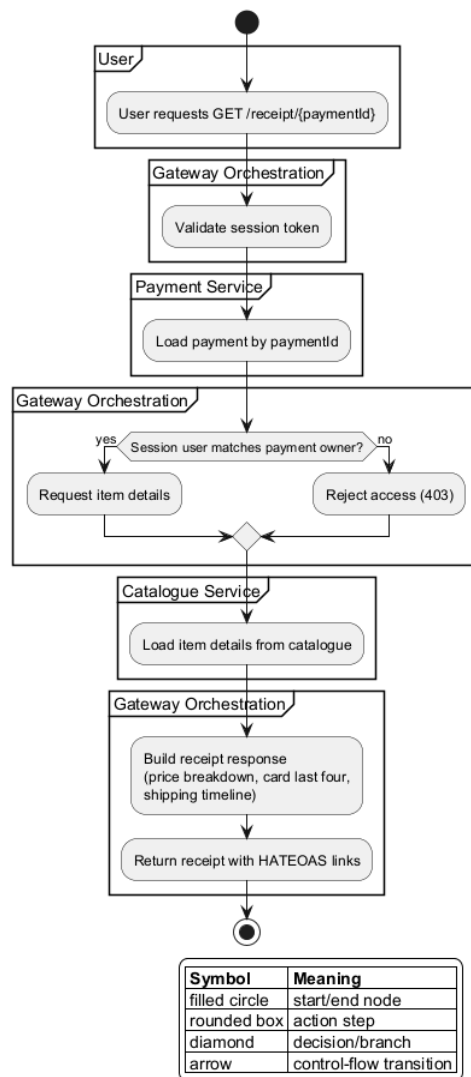
- Swimlanes separate scheduler/internal processing from user-facing result retrieval.
- Scheduler-driven end transition with SOLD/UNSOLD branch and catalogue update observer callback.



- **activity_uc5_payment_submit_d2:**

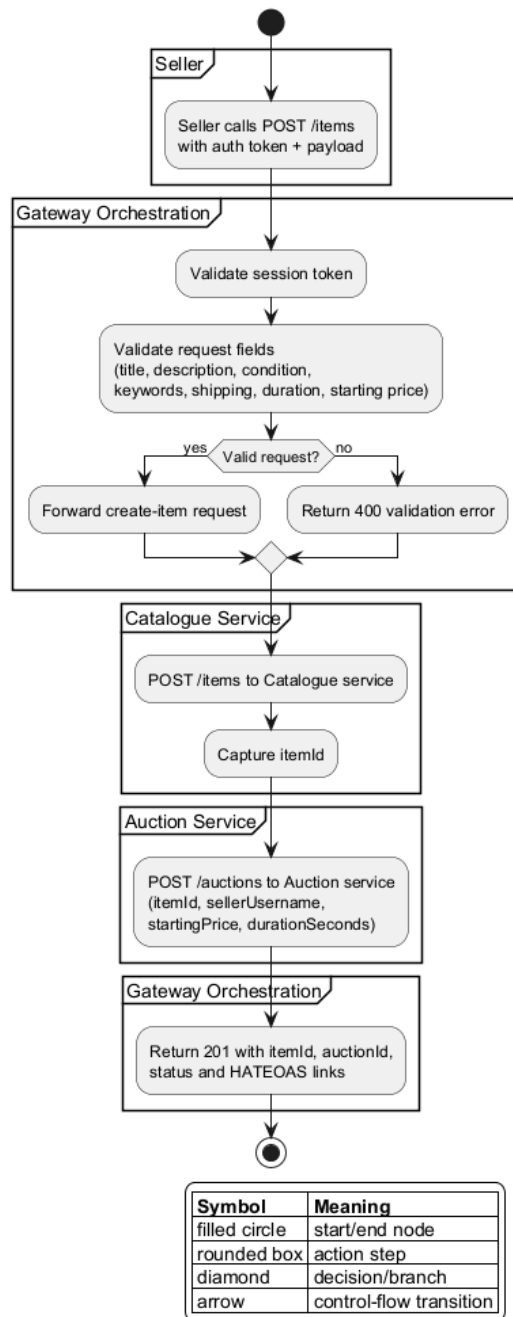
- Swimlanes separate user actions, gateway orchestration, and payment service responsibilities.
- Winner validation, card validation, duplicate guard, payment creation, and receipt-link return.

UC6 - Receipt Retrieval and Display (As Built)



- activity_uc6_receipt_d2 :
 - Swimlanes separate the user request path from gateway/payment/catalogue processing.
 - Owner check followed by receipt assembly and HATEOAS response links.

UC7 - Create Item and Auction (As Built)



- **Activity_uc7_create_item_d2:**

- Swimlanes separate seller action from gateway orchestration and downstream service operations.
- Two-step orchestration: create item in catalogue, then create auction with returned *itemId*.

20. AI Usage Acknowledgement (Advantages and Disadvantages)

Detailed acknowledgement is provided in: *AI_USAGE_ACKNOWLEDGEMENT.md*

For Deliverable 2, AI tools were used as a support aid during analysis, testing, and documentation. AI was not used as an autonomous decision-maker; instead, it was used to accelerate drafting and review tasks while final responsibility remained with the team.

20.1 Scope of AI Use

AI assistance was used to:

- Brainstorm edge cases and wrong-input scenarios for robustness testing,
- Suggest additional negative test ideas (validation failures, authorization failures, state-transition errors),
- Help identify potential requirement-coverage gaps between scripts, tests, and report content,
- Propose wording improvements for report sections and checklists,
- Improve documentation clarity (structure, consistency, and traceability formatting).

Examples of AI-assisted areas in this milestone:

- Robustness discussion around invalid inputs and corner-case behavior,
- Checklist-style mapping of rubric requirements to repository evidence,
- Wording improvements for architecture/pattern explanations,
- Review prompts for consistency between diagrams and implementation notes.

20.2 Human Oversight and Validation

- All final code, tests, diagrams, and documentation were reviewed by team members before inclusion.
- Any AI suggestions were treated as draft guidance and were accepted only after manual verification against:
 - source code behavior,
 - test outcomes,
 - milestone requirements and rubric criteria.

20.3 Advantages

- Faster discovery of potential edge cases and robustness scenarios.
- More efficient drafting of structured report sections and requirement matrices.
- Helpful second-pass review perspective for consistency and completeness checks.
- Reduced time spent on repetitive documentation formatting tasks.

20.4 Limitations / Risks

- AI suggestions may be incorrect, incomplete, or overly generic.
- AI may produce plausible wording that does not precisely match implementation details.
- Over-reliance could weaken technical precision if outputs are not independently validated.

20.5 Mitigation Measures

- Manual verification of all AI-assisted outputs against actual implementation artifacts.
- Preference for repository evidence (code/tests/scripts) over generated claims.
- Iterative review to align documentation with observed system behavior.
- Explicit retention of human authorship and approval for final submitted content.

21. Team Contribution:

<i>Team Members</i>	<i>Main Areas</i>	<i>Concrete Contributions</i>	<i>Verification Evidence</i>
Nathan Binu Edappilly (219317965)	Gateway service, project setup, launchers	Implemented all Gateway controllers (<i>AuthProxyController</i> , <i>CatalogueController</i> , <i>BidController</i> , <i>AuctionEndController</i> , <i>PaymentOrchestratorController</i> , <i>ItemOrchestratorController</i>). Built <i>SessionStore</i> for in-memory session management, <i>HateoasHelper</i> utility for HATEOAS link assembly, <i>HttpConfig</i> , and <i>GlobalExceptionHandler</i>. Authored all 8 Gateway test classes. Created <i>run_all.sh</i> and <i>run_all.bat</i> launcher scripts. Set up initial Maven multi-module project structure and <i>pom.xml</i> configurations.	<i>gateway/src/main/java/**</i> , <i>gateway/src/test/java/**</i> , <i>run_all.sh</i> , <i>run_all.bat</i> , <i>gateway/pom.xml</i>
Umer Shaikh (218931790)	Auction service, design patterns, concurrency	Implemented <i>AuctionController</i> , <i>AuctionService</i>, and <i>AuctionScheduler</i> (scheduled auction-end polling). Designed and implemented the Strategy pattern (<i>BidValidationStrategy</i>,	<i>auction/src/main/java/**</i> , <i>auction/src/test/java/**</i> ,

		<i>IntegerBidValidator</i> , <i>IncreasingBidValidator</i>) and Observer pattern (<i>AuctionEndObserver</i> , <i>CatalogueNotifier</i>). Built <i>AuctionRepository</i> with atomic <i>tryUpdateBidIfHigher</i> for concurrency safety, and BidRepository. Authored all 4 Auction test classes including <i>AuctionServiceConcurrencyTest</i>. Created Auction schema and DTOs.	<i>auction/</i> <i>src/main</i> <i>/resources/schema.sql</i>
Imaduddin Ahmed (219569029)	IAM service, Catalogue service, UML diagrams	Implemented IAM service: <i>AuthController</i>, <i>AuthService</i> (BCrypt hashing, login, resetpassword), <i>UserRepository</i>, and all IAM DTOs. Implemented Catalogue service: <i>ItemController</i>, <i>ItemRepository</i> (keyword search, status updates), and all Catalogue DTOs. Authored IAM tests (4 classes) and Catalogue tests (3 classes). Designed all UML diagrams: component diagram, class diagrams (auction + gateway subsystems), sequence diagrams (UC1, UC2, UC3, UC5/UC6), and activity diagrams (UC4, UC5, UC6, UC7). Created IAM and Catalogue schemas.	<i>iam/src/</i> <i>main/java</i> <i>/**</i> , <i>catalogue/src/main/java/</i> <i>**</i> , <i>iam/src/</i> <i>test/java</i> <i>/**</i> , <i>catalogue/src/test/java/</i> <i>**</i> , <i>docs/diagrams/*.</i> <i>png</i>
Pawanjot Randhawa (219732718)	Payment service, scripts, documentation	Implemented Payment service: <i>PaymentController</i>, <i>PaymentService</i> (card validation, per-auction <i>ReentrantLock</i> concurrency control), <i>PaymentRepository</i> (idempotent insert). Authored all 3 Payment test classes including <i>PaymentServiceConcurrencyTest</i>. Created all shell scripts: main-flow suite (<i>full_flow.sh</i>, <i>00_smoke.sh</i>, <i>01_signup.sh</i>, <i>02_login.sh</i>, <i>03_uc7_create_item.sh</i>) and robustness suite (<i>test_robustness.sh</i> + 6 individual	<i>payment/</i> <i>src/main</i> <i>/java/**</i> , <i>payment/</i> <i>src/test</i> <i>/java/**</i> , <i>scripts</i> <i>/**</i> , <i>README.</i> <i>md</i> , <i>TESTING_</i> <i>INSTRUCT</i> <i>IONS.md</i> , <i>AI_USAGE</i> <i>_ACKNOWLEDGEMENT.md</i>

		robustness scripts). Wrote <i>README.md</i>, <i>TESTING_INSTRUCTIONS.md</i>, <i>AI_USAGE_ACKNOWLEDGEMENT.md</i>, <i>DELIVERABLE_2_REQUIREMENTS_MATRIX.md</i>, and compiled the Deliverable 2 Report. Created <i>seed_data.sql</i>.	<i>EDGEMENT.md</i>
--	--	--	---------------------------

22. Team Meeting Logs:

Meeting Date	Friday, February 20, 2026
Attendance	Nathan Binu Edappilly, Umer Shaikh, Imaduddin Ahmed, Pawanjot Randhawa (All Present)
Topics Discussed	1. Review of Deliverable 2 instructions and marking rubric. 2. Revisited Milestone 1 architecture decisions and identified areas requiring changes for implementation. 3. Confirmed five-service decomposition: Gateway, IAM, Catalogue, Auction, Payment. 4. Discussed technology stack: Spring Boot, Java 17, SQLite per service, Spring JDBC. 5. Discussed design pattern requirements (minimum 4 used, 2 considered-but-not-used).
Decisions Made	● Architecture: Finalized gateway-facade plus four backend services with service-owned SQLite databases, moving away from the Milestone 1 layered/monolithic baseline. ● Pattern assignments: Facade (Gateway), Repository (all services), Strategy (Auction bid validation), Observer (Auction end side effects). ● Considered-but-not-used: Abstract Factory and Manual Singleton. ● Each team member owns one primary service or equivalent workload.
Action Items	● Nathan: Initialize Maven multi-module project structure; begin Gateway service scaffolding (controllers, session management).

	• Umer: Set up Auction service with controller, service layer, repository, and schema. Design Strategy and Observer pattern interfaces. • Imaduddin: Implement IAM service (signup, login, reset-password with BCrypt) and Catalogue service (item CRUD, keyword search). Create schemas for both. • Pawanjot: Implement Payment service (payment submission, card validation, receipt retrieval). Begin drafting README and TESTING_INSTRUCTIONS.
--	--

Meeting Date	Friday, February 27, 2026
Attendance	Nathan Binu Edappilly, Umer Shaikh, Imaduddin Ahmed, Pawanjot Randhawa (All Present)
Topics Discussed	1. Demo of each service running independently, verified all five services start on ports 8080 to 8084. 2. Reviewed gateway orchestration flows for UC1 through UC7. 3. Discussed HATEOAS link strategy: which endpoints need _links and what next-step navigation to include. 4. Reviewed concurrency concerns: concurrent bids on same auction, duplicate payment prevention. 5. Planned testing approach: module-level JUnit tests, end-to-end shell scripts, robustness scripts. 6. Discussed validation gaps found during initial integration (missing keywords field validation in UC7, bid amount integer enforcement).
Decisions Made	• Gateway will add _links on all business-flow responses (auth, selection, bid, payment, receipt) using a shared HateoasHelper utility. • The auction service will use atomic SQL (tryUpdateBidIfHigher) plus Strategy validators for bid concurrency safety. • Payment service will use per-auction ReentrantLock plus idempotent SQL insert (WHERE NOT EXISTS) to prevent duplicate payments. • Added @NotBlank on keywords in UC7 DTO and integer-only enforcement for bid amounts in Gateway. • Agreed on test responsibilities: each member writes tests for their owned service; Pawanjot additionally handles integration shell scripts.

Action Items:	• Nathan: Implement <i>HateoasHelper</i>, add HATEOAS links to all gateway controllers, and implement <i>GlobalExceptionHandler</i> with downstream error passthrough. Write Gateway unit tests. • Umer: Implement <i>AuctionScheduler</i> (5-second polling), <i>CatalogueNotifier</i> observer, <i>IntegerBidValidator</i>, and <i>IncreasingBidValidator</i> strategies. Write <i>AuctionServiceConcurrencyTest</i>. • Imaduddin: Finalize IAM extended tests (reset-password, profile lookups) and Catalogue validation tests. Begin PlantUML diagrams (component, class, sequence, activity). • Pawanjot: Implement <i>PaymentServiceConcurrencyTest</i>. Write <i>full_flow.sh</i> end-to-end script and <i>test_robustness.sh</i> automated robustness suite.
----------------------	---

Meeting Date	Monday, March 2, 2026
Attendance	Nathan Binu Edappilly, Umer Shaikh, Imaduddin Ahmed, Pawanjot Randhawa (All Present)
Topics Discussed	1. End-to-end integration test using <i>full_flow.sh</i> — identified and fixed remaining issues (session token propagation, auction-end timing in script). 2. Reviewed robustness test results: 15+ edge cases across UC1/UC2/UC3/UC5/UC7 passing. 3. Reviewed all 11 PlantUML diagrams for consistency with as-built code. 4. Discussed report structure: deviation map from Milestone 1, alternative architecture comparison, coupling/cohesion justification. 5. Reviewed <i>run_all.sh/run_all.bat</i> launcher scripts for cross-platform support.
Decisions Made	• Full-flow script confirmed working end-to-end with 30-second auction duration for demo. • All diagrams approved; minor label fixes applied to sequence diagrams for consistency with controller names. • Report sections assigned for final write-up. • Decided to include explicit "real-time scope" clarification in report (no WebSockets/push in D2)

Action Items	• Nathan: Finalize run_all.sh and run_all.bat with graceful shutdown and port cleanup. Perform final review of Gateway tests. • Umer: "Write report sections: 2 (Deviations), 5 (Design Patterns Used), and 6 (Patterns Considered but Not Used)." • Imaduddin: "Export final diagram PNGs. Write report sections: 3 (Alternative Architecture), 4 (Coupling/Cohesion), and 9 (Diagram Inventory)." • Pawanjot: "Finalize all scripts, README, and TESTING_INSTRUCTIONS. Write report sections: 7 (Technology Choices) and 8 (Test Completeness Matrix)."
---------------------	--

Meeting Date	Wednesday, March 4, 2026
Attendance	Nathan Binu Edappilly, Umer Shaikh, Imaduddin Ahmed, Pawanjot Randhawa (All Present)
Topics Discussed	1. Final review of complete Deliverable 2 report draft. 2. Ran mvn test on all five services - confirmed all tests passing. 3. Ran full_flow.sh and test_robustness.sh one final time on a clean build (deleted all *.db files first). 4. Verified ZIP packaging excludes logs/, *.db, Context/, and target/ directories. 5. Cross-checked report writing quality: consistent terminology, diagram legends, reference completeness.
Decisions Made	• Report finalized. All sections complete and cross-referenced. • AI usage acknowledgement reviewed and agreed upon by all members. • Team contribution table confirmed as equal 25% split. • Submission package structure confirmed: source code, scripts, diagrams, documentation, report.
Action Items	• Nathan: Final proofread of report sections 1, 5, 6, and 10 (AI acknowledgement summary). • Umer: Final proofread of report sections 2, 3, 4. Verify all code references in the report match actual file paths. • Imaduddin: Final proofread of sections 9, 11, 12. Verify diagram file list matches docs/diagrams/ Pawanjot: Compile final ZIP package. Final proofread of sections 7, 8, 13, 14. Submit deliverable.

23. Mark Alignment

23.1 Implementation Mark (/80) Alignment:

Rubric Dimension	Implementation Evidence	Verification Entry Points
Use cases implemented on server per specification	UC1-UC7 flows implemented across <i>gateway/</i> , <i>iam/</i> , <i>catalogue/</i> , <i>auction/</i> , <i>payment/</i>	<i>scripts/main_flow/full_flow.sh</i> , gateway API endpoints in <i>README.md</i>
Robustness to wrong user inputs	Validation + exception handling in controllers/services; robustness scripts	<i>scripts/robustness/test_robustness.sh</i> , <i>*/src/test/java/**/Validation*Test.java</i>
Easy to install/run/test	One-command startup scripts and explicit test instructions	<i>run_all.sh</i> , <i>run_all.bat</i> , <i>TESTING_INSTRUCTIONS.md</i> , <i>README.md</i>
Code organization/style/pattern usage	Layered package structure + documented Facade/Repository/Strategy/Observer use	Sections 11,13,14,19
REST principles + HATEOAS	Gateway responses include <i>_links</i> on business-flow endpoints	<i>gateway/src/main/java/**</i> , <i>gateway/src/test/java/**</i> , <i>README.md</i> API examples
Back-end test coverage	Module test suites + concurrency regressions + scripted robustness	<i>*/src/test/java/</i> , <i>auction/.../AuctionServiceConcurrencyTest</i> , <i>payment/.../PaymentServiceConcurrencyTest</i>

23.2 Report Content Mark (/15) Alignment:

Rubric Dimension	Where Addressed in Deliverable 2
Changes to architecture/design/patterns and implementation consistency	Sections 11, 12, 13, 14, 15, 16, 17

Technology choices and justification	Section 16
Class/sequence/subsystem diagrams	Section 19 + PNG exports in <i>docs/diagrams/</i> with legends and updated layouts
Test-case completeness	Section 17 + scripts/tests references
AI usage (advantages/disadvantages)	Section 20 + <i>AI_USAGE_ACKNOWLEDGEMENT.md</i>
Team contributions documented	Section 21

23.3 Report Writing Quality Mark (/5) Controls:

Rubric Dimension	Control Used in This Package
Consistency of labels/legends	Unified naming in section 19 legends embedded in all diagrams
Diagram/table explanation in text	Section 19 walkthrough for each figure + Section 18 for Use Case / Scope
References present	Section 24 references list
Proofreading readiness	Section 23 checklist used as pre-submission gate
Readable UML screenshots/layout quality	Section 19 + PNG exports in <i>docs/diagrams/</i> with legends and updated layouts

24. References

24.1 Academic & Course Materials

1. York University. EECS 4413 Deliverable 2: Backend Implementation and Testing Instructions. Course Handout, Winter 2026.
2. Team 5. Milestone 1 Software Design Document. Submitted February 2026 (External Artifact).

24.2 Technical Frameworks & Security

3. Spring Framework Documentation. "Spring Boot Reference Guide."
<https://docs.spring.io/spring-boot/documentation.html>
4. Provos, N., & Mazières, D. "BCrypt Password Hashing Function."
<https://www.mindrot.org/projects/jBCrypt/>

24.3 Design Patterns & Modeling

5. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. (Reference for Facade, Repository, Strategy, and Observer patterns).
6. Object Management Group (OMG). Unified Modeling Language (UML) Specification. v2.5.1. <https://www.omg.org/spec/UML/>

24.4 Project Artifacts & Scripts

7. PrimeBid Project. *README.md* (System Entry Point and API Documentation).
8. PrimeBid Project. *scripts/main_flow/full_flow.sh* (End-to-End Use Case Verification Suite).
9. PrimeBid Project. *scripts/robustness/test_robustness.sh* (Negative Testing and Error Handling Suite).
10. PrimeBid Project. *scripts/seed_data.sql* (Database Initialization and Test State Injection).